

פעולות



ביחידה זו נלמד :

- מהי פעולה ומיקומה במחלקה.
- פעולות ממחלקות אחרות (קופסא שחורה).
- טווח הכרה של משתנים.
- מחסנית הקריאות.
- פרמטרים של פעולה.
- פעולה המקבלת יותר מפרמטר אחד, כולל תוכנית לדוגמא.
- פעולה המחזירה ערך, כולל תוכנית לדוגמא.
- כיצד נראה הזיכרון.
- העמסת פעולות.
- תכנון Top-Down בשילוב פעולות כולל דוגמאות.
- שאלת סיכום.
- העברת פרמטרים.
- **העשרה:** פרמטרי ref ופרמטרי out.

מחלקה - אוסף של פעולות (*ותכונות – בהמשך)

```
using System;  
  
namespace StaticMethod  
{
```

```
    class Program
```

```
    {
```

```
        static void PrintHello()
```

```
        {
```

```
            Console.WriteLine("hello");
```

```
        }
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Console.WriteLine("Hello World!");
```

```
        }
```

```
    }
```

```
}
```

הפעולה PrintHello

הפעולה Main

מחלקה - אוסף של פעולות

```
using System;

namespace StaticMethod
{
    class Program
    {
        static void PrintHello()
        {
            Console.WriteLine("hello");
        }

        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

פעולה היא :
אוסף של הוראות לביצוע
המאוגדות יחד,
שניתן להן שם

מחלקה - אוסף של פעולות

```
using System;

namespace StaticMethod
{
    class Program
    {
        static void PrintHello()
        {
            Console.WriteLine("hello");
        }

        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

הפעולה הראשית **Main**, היא מיוחדת –
תפקידה לנהל את זרימת התוכנית.

הרצת התוכנית מתחילה תמיד ב-**Main**.
והיא אשר קובעת את סדר ביצוע התוכנית.

מחלקה - אוסף של פעולות

```
using System;

namespace StaticMethod
{
    class Program
    {
        static void PrintHello()
        {
            Console.WriteLine("hello");
        }

        static void Main(string[] args)
        {
            PrintHello();
        }
    }
}
```

נוסיף למחלקה פעולות, כרצוננו,
עבור ביצוע מטרות ספציפיות.

פעולות אלו תתבצעה
רק אם נזמן אותן לביצוע.

שימוש בפעולות מובנות

```
static void Main(string[] args)
{
    Console.Write("enter a number: ");

    int n1 = int.Parse(Console.ReadLine());

    int n2 = Math.Abs(n1);

    Console.WriteLine(n1 + " " + n2);

    Console.WriteLine();

    Random rnd = new Random();

    int n3 = rnd.Next(5, 9);

    int n4 = rnd.Next(10);

    Console.WriteLine(n1 + " " + n2 + " " + n3 + " " + n4);
}
```

הקוד כאן מכיל:

- את הגדרת הפעולה **Main**.
- עשרה זימונים לפעולות מובנות, השייכות למחלקות שונות.

שימוש בפעולות מובנות

הפעולה הראשית

```
static void Main(string[] args)
{
```

פעולה ממחלקת Int32

```
    Console.Write("enter a number: ");
```

```
    int n1 = int.Parse(Console.ReadLine());
```

פעולה ממחלקת Math

```
    int n2 = Math.Abs(n1);
```

```
    Console.WriteLine(n1 + " " + n2);
```

```
    Console.WriteLine();
```

```
    Random rnd = new Random();
```

פעולות ממחלקת Random

```
    int n3 = rnd.Next(5, 9);
```

```
    int n4 = rnd.Next(10);
```

```
    Console.WriteLine(n1 + " " + n2 + " " + n3 + " " + n4);
```

```
}
```

פעולות ממחלקת Console

פעולות ממחלקות אחרות - קופסא שחורה

```
static void Main(string[] args)
{
    Console.Write("enter a number: ");

    int n1 = int.Parse(Console.ReadLine());

    int n2 = Math.Abs(n1);

    Console.WriteLine(n1 + " " + n2);

    Console.WriteLine();

    Random rnd = new Random();

    int n3 = rnd.Next(5, 9);

    int n4 = rnd.Next(10);

    Console.WriteLine(n1 + " " + n2 + " " + n3 + " " + n4);
}
```

כדי להשתמש (לזמן) פעולה
צריך לדעת :
- את שם המחלקה שהפעולה שייכת אליה,
- את שם הפעולה,
- האם ואילו נתונים הפעולה צריכה לקבל.
- ומה הפעולה עושה.

פעולות ממחלקות אחרות - קופסא שחורה

לגבי הפעולה **Main** לא מדובר
בזימון אלא בהגדרת הפעולה.
לכן יש פירוט של טיפוס הערך
שהפעולה מצפה לקבל כ**נתון**.

אם הפעולה זקוקה ל**נתונים**,
היא **מקבלת** אותם מהערכים
הנשלחים בזימון.

הפעולה **Parse** מקבלת
את הנתון המוחזר
מהפעולה **ReadLine**

```
static void Main(string[] args)
{
    Console.Write("enter a number: ");

    int n1 = int.Parse(Console.ReadLine());

    int n2 = Math.Abs(n1);

    Console.WriteLine(n1 + " " + n2);

    Console.WriteLine();

    Random rnd = new Random();

    int n3 = rnd.Next(5, 9);

    int n4 = rnd.Next(10);

    Console.WriteLine(n1 + " " + n2 + " " + n3 + " " + n4);
}
```

כל הפעולות שכתבנו עד היום לא נזקקו לנתונים חיצוניים !

פעולות ממחלקות אחרות -

קופסא שחורה

```
static void Main(string[] args)
{
    Console.Write("enter a number: ");

    int n1 = int.Parse(Console.ReadLine());

    int n2 = Math.Abs(n1);

    Console.WriteLine(n1 + " " + n2);

    Console.WriteLine();

    Random rnd = new Random();

    int n3 = rnd.Next(5, 9);

    int n4 = rnd.Next(10);

    Console.WriteLine(n1 + " " + n2 + " " + n3 + " " + n4);
}
```

בדימון פעולות ממחלקה אחרת,
כתבנו גם את שם המחלקה.
**מחלקות שונות יכולות להכיל
פעולות בשם זהה !**

בפעולות הנמצאות באותה מחלקה
(כמו הפעולות שכתבנו עד היום),
אין חובה לכתוב את שם המחלקה.
ברירת המחדל היא פעולה
שנמצאת במחלקה עצמה.

פעולות ממחלקות אחרות - קופסא שחורה

```
static void Main(string[] args)
{
    Console.WriteLine("enter a number: ");

    int n1 = int.Parse(Console.ReadLine());

    int n2 = Math.Abs(n1);

    Console.WriteLine(n1 + " " + n2);

    Console.WriteLine();

    Random rnd = new Random();

    int n3 = rnd.Next(5, 9);

    int n4 = rnd.Next(10);

    Console.WriteLine(n1 + " " + n2 + " " + n3 + " " + n4);
}
```

כדי להגריל מספרים רנדומליים:
- נבנה קודם עצם מטיפוס Random,
- ואח"כ נפעיל את הפעולה על העצם !

- פעולות שאינן פועלות על עצם הן פעולות סטטיות .static
- פעולות על עצמים אינן סטטיות !

פעולות ממחלקות אחרות - קופסא שחורה

```
static void Main(string[] args)
{
    Console.Write("enter a number: ");

    int n1 = int.Parse(Console.ReadLine());

    int n2 = Math.Abs(n1);

    Console.WriteLine(n1 + " " + n2);

    Console.WriteLine();

    Random rnd = new Random();

    int n3 = rnd.Next(5, 9);

    int n4 = rnd.Next(10);

    Console.WriteLine(n1 + " " + n2 + " " + n3 + " " + n4);
}
```

חלק מהפעולות החזירו ערכים
בהם השתמשנו אחר-כך:
- בפקודות השמה למשתנה או
- כנתון לפעולה אחרת.

פעולות שאינן מחזירות ערך,
כמו ה-Main והפעולות שכתבנו עד היום
הן פעולות void.

מהי פעולה ? סיכום חלקי

- פעולה היא אוסף הוראות לביצוע שמממשות רעיון משותף.
- השתמשנו בעבר בפעולות: `Math.pow`, `Console.WriteLine` ועוד.
גם ה-`Main` שאנחנו כותבים הוא **פעולה** (הפעולה הראשית).
- פעולות ממחלקה אחרת בהן השתמשנו הן "קופסה שחורה" עבורנו.
 - אנו יודעים **מה** הן עושות,
 - אבל לא יודעים **איך**,
 - כלומר מהו אוסף הפעולות המבוצעות בהן.



```

static void Main(string[] args)
{
    for (int i=0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
            Console.Write("*");
        Console.WriteLine();
    }

    Console.WriteLine();

    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
            Console.Write("*");
        Console.WriteLine();
    }
}

```

דוגמה לשכפול קוד : ציור ריבוע

ציור ריבוע בגודל 3X3

```

***
***
***

****
****
****
****

```

ציור ריבוע בגודל 4X4

רצף הפעולות לציור ריבוע זהה.

בכל פעם ביצענו את הפעולות על ערכים שונים ← שכפול קוד !
מי שקורא את ה-Main צריך להבין מה תפקידה של כל לולאה.

ציור ריבוע : ללא שכפול קוד

שם הפעולה יעיד מה היא עושה

```
static void DrawSquare(int size)
```

```
{
```

```
    for (int i = 0; i < size; i++)
```

```
    {
```

```
        for (int j = 0; j < size ; j++)
```

```
            Console.WriteLine("*");
```

```
            Console.WriteLine();
```

```
    }
```

```
}
```

הנתון שהפעולה צריכה לקבל, נקרא פרמטר

קוד הפעולה
לביצוע המשימה

```
static void Main(string[] args)
```

```
{
```

```
    DrawSquare(3);
```

```
    Console.WriteLine();
```

```
    DrawSquare(4);
```

```
}
```

זימון פעולה כדי לצייר ריבוע בגודל 3X3

זימון פעולה כדי לצייר ריבוע בגודל 4X4

ציור ריבוע : רצף הפקודות בהרצה

```
static void DrawSquare(int3 size)
{
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size ; j++)
            Console.Write("*");
        Console.WriteLine();
    }
} // סיום הפעולה

static void Main(string[] args)
{
    DrawSquare(3);
    Console.WriteLine();
    DrawSquare(4);
}
```

הערך ששולחים לפעולה חייב להיות מהטיפוס המבוקש.

ציור ריבוע : רצף הפקודות בהרצה

```
***  
***  
***
```

```
static void DrawSquare(int size)  
{  
    for (int i = 0; i < size; i++)  
    {  
        for (int j = 0; j < size ; j++)  
            Console.Write("*");  
        Console.WriteLine();  
    }  
} // סיום הפעולה  
  
static void Main(string[] args)  
{  
    DrawSquare(3);  
    Console.WriteLine();  
    DrawSquare(4);  
}
```

ציור ריבוע : רצף הפקודות בהרצה

```
static void DrawSquare(int size4)
{
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size ; j++)
            Console.WriteLine("*");
    }
} //סיום הפעולה

static void Main(string[] args)
{
    DrawSquare(3);
    Console.WriteLine();
    DrawSquare(4);
}
```

```
***
***
***

****
****
****
****
```

ציור ריבוע : רצף הפקודות בהרצה

```
static void DrawSquare(int size)
{
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size ; j++)
            Console.Write("*");
        Console.WriteLine();
    }
} // סיום הפעולה

static void Main(string[] args)
{
    DrawSquare(3);
    Console.WriteLine();
    DrawSquare(4);
}
```

```
***
***
***

****
****
****
****
```

השימוש בפעולה יוצר קוד קצר וקריא יותר !
ה-Main כתוב ב"ראשי פרקים"
וניתן להבין בקלות מה הוא מבצע.


```
static void VariablesScope(int a)
{
    int x = 8, y = 2;

    if (x < 10)
    {
        int z = 5;
    }

    if (y < 10)
    {
        int z = 7;
    }
    Console.WriteLine("a=" + a);
}
```

טווח הכרה של משתנים

- משתנה מוגדר ומוכר אך ורק **בבלוק בו הוגדר** (ובבלוקים הפנימיים לו).
 ✓ פרמטר הוא משתנה שהוגדר בשורה הראשונה של הפעולה ולכן יוכר **בכל** הפעולה.
 ✓ משתנה מקומי יוכר משורת ההגדרה שלו ועד לסוף הבלוק בו הוא הוגדר.
- שם משתנה **חייב** להיות **ייחודי** בבלוק.

```

static void VariablesScope(int a)
{
    int x = 8, y = 2;

    if (x < 10)
    {
        int z = 5;
        int y = 3;
    }

    if (y < 10)
    {
        int z = 7;
    }
    Console.WriteLine("a=" + a);
}

```

טווח הכרה של משתנים

- משתנה מוגדר ומוכר אך ורק **בבלוק בו הוגדר** (ובבלוקים הפנימיים לו).
 ✓ פרמטר הוא משתנה שהוגדר בשורה הראשונה של הפעולה
 ולכן יוכר בכל הפעולה.
 ✓ משתנה מקומי יוכר משורת ההגדרה שלו
 ועד לסוף הבלוק בו הוא הוגדר.
- שם משתנה **חייב** להיות **ייחודי** בבלוק.

טווח הכרה של משתנים

```
static void VariablesScope(int a)
{
    int x = 8, y = 2;

    if (x < 10)
    {
        int z = 5;
        int y = 3;
    }

    if (y < 10)
    {
        int z = 7;
    }
}
```

משתנה המוגדר בתוך בלוק מוכר :
- בבלוק עצמו
- ובבלוקים הפנימיים לו

טווח הכרה של משתנים

```
static void VariablesScope(int a)
{
    int x = 8, y = 2;

    if (x < 10)
    {
        int z = 5;
        int y = 3;
    }

    if (y < 10)
    {
        int z = 7;
    }
}
```

A local or parameter named 'y' cannot be declared in this scope because that name is used in an enclosing local scope to define a local or parameter

לא ניתן להגדיר משתנים בעלי שם זהה
בבלוקים מקוננים

משתנה המוגדר בתוך בלוק מוכר :
- בבלוק עצמו
- ובבלוקים הפנימיים לו

טווח הכרה של משתנים

```
static void VariablesScope(int a)
{
    int x = 8, y = 2;

    if (x < 10)
    {
        int z = 5;
        int y = 3;
    }

    if (y < 10)
    {
        int z = 7;
    }
}
```

לא ניתן להגדיר משתנים בעלי שם זהה
בבלוקים מקוננים

A local or parameter named 'y' cannot be declared in this scope because that name is used in an enclosing local scope to define a local or parameter

משתנה המוגדר בתוך בלוק מוכר :
- בבלוק עצמו
- ובבלוקים הפנימיים לו

טווח הכרה של משתנים

```
static void VariablesScope(int a)
{
    int x = 8, y = 2;

    if (x < 10)
    {
        int z = 5;
        int y = 3;
    }

    if (y < 10)
    {
        int z = 7;
    }
    Console.WriteLine("z=" + z);
}
```

לא ניתן להגדיר משתנים בעלי שם זהה
בבלוקים מקוננים

A local or parameter named 'y' cannot be declared in this scope because that name is used in an enclosing local scope to define a local or parameter

ניתן להגדיר משתנים בעלי שם זהה
בבלוקים עוקבים

The name 'z' does not exist in the current context

משתנה המוגדר בתוך בלוק מוכר :
- בבלוק עצמו
- ובבלוקים הפנימיים לו

טווח הכרה של משתנים

```
static void VariablesScope(int a)
{
    int x = 8, y = 2;

    if (x < 10)
    {
        int z = 5;
        int y = 3;
    }

    if (y < 10)
    {
        int z = 7;
    }
    Console.WriteLine("z=" + z);
}
```

לא ניתן להגדיר משתנים בעלי שם זהה
בבלוקים מקוננים

A local or parameter named 'y' cannot be declared in this scope because that name is used in an enclosing local scope to define a local or parameter

ניתן להגדיר משתנים בעלי שם זהה
בבלוקים עוקבים

The name 'z' does not exist in the current context

משתנה המוגדר בתוך בלוק
מוכר אך ורק בבלוקים הפנימיים

משתנה המוגדר בתוך בלוק מוכר :
- בבלוק עצמו
- ובבלוקים הפנימיים לו

Stack overflow.

```
static int foo()  
{  
    return foo();  
}
```

Exception Unhandled

System.StackOverflowException: "

מחסנית (stack) הקריאות

- מחסנית היא מבנה השומר נתונים

- הערך הראשון שנכנס הוא האחרון שיוצא.
- הערך האחרון שהוכנס הוא הראשון שיוצא.



- ראינו:

- כאשר קוראים (מזמנים) לפעולה, "קופצים" לאוסף ההוראות שלה.
- כאשר הפעולה מסתיימת, חוזרים לביצוע הפעולות מהמקום בו קראנו לה.
- ← הפעולה הראשונה שהתחלנו (ה-Main) היא האחרונה להסתיים.

- למנגנון השומר את סדר הפעלת הפעולות קוראים **מחסנית קריאות** או **מחסנית זמן ריצה**.

- כאשר מחסנית הקריאות מתמלאת, ואין בה מקום, מתקבלת השגיאה Stack overflow.

מחסנית הקריאות : דוגמה

```
static void DrawSquare(int size)
{
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size ; j++)
            Console.Write("*");
        Console.WriteLine();
    }
}

static void Main(string[] args)
{
    DrawSquare(3);
    Console.WriteLine();
    DrawSquare(4);
}
```



DrawSquare(3)

Main
(line 1)

מחסנית הקריאות

מחסנית הקריאות : דוגמה

```
static void DrawSquare(int size)
{
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size ; j++)
            Console.Write("*");
        Console.WriteLine();
    }
}

static void Main(string[] args)
{
    DrawSquare(3);
    Console.WriteLine();
    DrawSquare(4);
}
```

DrawSquare(4)

Main

מחסנית הקריאות

כתיבת פעולה : בחירת השם

~~Draw~~

DrawSquare

- שם הפעולה צריך להעיד מה היא עושה

- מבחינת סגנון כתיבה ההמלצות הן :

- שם הפעולה יתחיל ✓

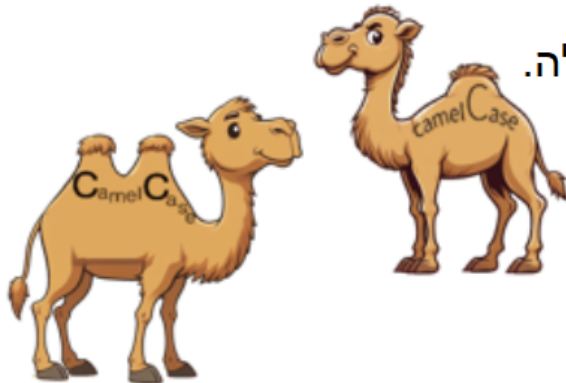
- באות גדולה, אם היא public

- באות קטנה, אם היא private

- ✓ אם יש יותר ממילה אחת בשם הפעולה, כל מילה נוספת תתחיל באות גדולה.

סגנון זה נקרא Camel Case

- זהו סגנון הכתיבה המקובל.



פעולה המקבלת יותר מפרמטר אחד

```
static void DrawRectangle(int width, int height, char ch)
{
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
            Console.WriteLine(ch);
    }
}
```

רשימת הפרמטרים שהפעולה תקבל יופרדו ע"י פסיק,
ויש לציין את הטיפוס לפני כל פרמטר, גם אם כולם מאותו סוג.

```
static void Main(string[] args)
{
    DrawRectangle(3, 2, '#');
}
```

סדר שליחת הערכים קריטי ומשפיע על התוצאה.
יש לשלוח ערכים כמספר הפרמטרים ומהסוג המבוקש.

###

פעולה המקבלת יותר מפרמטר אחד

```
static void DrawRectangle(int width, int height, char ch)
{
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
            Console.WriteLine(ch);
    }
}
```

רשימת הפרמטרים שהפעולה תקבל יופרדו ע"י פסיק,
ויש לציין את הטיפוס לפני כל פרמטר, גם אם כולם מאותו סוג.

```
static void Main(string[] args)
{
    DrawRectangle(3, 2, '#');
    Console.WriteLine();
    DrawRectangle(2, 3, '@');
}
```

סדר שליחת הערכים קריטי ומשפיע על התוצאה.
יש לשלוח ערכים **כמספר** הפרמטרים ו**מהסוג** המבוקש.

###

###

@@

@@

@@

כתיבה ושימוש בפעולה : הזמנת מוצרים

בהזמנה טלפונית, העלות הכוללת לקנייה. יש להוסיף למחיר המוצר את עלות המשלוח, כאשר משלוח רגיל (Standard) עולה 5 ₪ ומשלוח אקספרס (Express) עולה 10 ₪. אם המזמין הוא חבר מועדון, הוא יקבל הנחה של 2 ₪.

כתבו פעולה המקבלת את מחיר המוצר, את סוג המשלוח ומשתנה שמשמעותו האם חבר מועדון.

הפעולה תדפיס את העלות הכוללת לקנייה.

```
static void CalculateShippingCost(double totalCost, string shippingMethod, bool isMember)
```

הפעולה תדפיס :
* אין ערך החזרה

הפעולה מקבלת 3 פרמטרים :
* מחיר המוצר
* סוג המשלוח
* האם חבר מועדון ?

כתיבה ושימוש בפעולה : הזמנת מוצרים

בהזמנה טלפונית, העלות הכוללת לקנייה. יש להוסיף למחיר המוצר את עלות המשלוח, כאשר משלוח רגיל (Standard) עולה 5 ₪ ומשלוח אקספרס (Express) עולה 10 ₪. אם המזמין הוא חבר מועדון, הוא יקבל הנחה של 2 ₪.

כתבו פעולה המקבלת את מחיר המוצר, את סוג המשלוח ומשתנה שמשמעותו האם חבר מועדון. הפעולה תדפיס את העלות הכוללת לקנייה.

```
static void CalculateShippingCost(double totalCost, string shippingMethod, bool isMember)
{
    double shippingCost = 0.0;

    if (shippingMethod == "Express")
        shippingCost = 10.0;
    else if (shippingMethod == "Standard")
        shippingCost = 5.0;

    if (isMember)
        shippingCost -= 2.0;

    Console.WriteLine("Total Cost: " + (totalCost + shippingCost));
}
```

כתיבה ושימוש בפעולה : הזמנת מוצרים

כתבו קטע קוד ב-Main אשר מדפיס את העלות למשלוח של חמש הזמנות של חמישה לקוחות. הקטע יגריל מחיר רנדומלי בין 10 ש"ל ל-101 ש"ל, יקלוט את סוג המשלוח (Standard / Express) וכן האם הלקוח חבר במועדון. יש להשתמש בפעולה CalculateShippingCost.

```
Random rand = new Random();
for (int i = 0; i < 5; i++)
{
    double totalCost = rand.Next(1000, 10001)/100.0;

    Console.WriteLine("Purchase #" + (i + 1));
    Console.WriteLine("Total Cost: " + totalCost);

    Console.Write("Enter shipping method (Express/Standard): ");
    string shippingMethod = Console.ReadLine();

    Console.Write("Is the customer a club member? (true/false): ");
    bool isMember = bool.Parse(Console.ReadLine());

    CalculateShippingCost(totalCost, shippingMethod, isMember);
    Console.WriteLine();
}
```

כדי להגריל מספר עשרוני בטווח 10-101, נגריל מספר הגדול פי 100 (בטווח שבין 1000 ל-10000 כולל) ונחלק ב-100.0. כך נקבל מספר בטווח הנכון עם 2 מקומות אחרי הנקודה.


```
Total Cost: 58.46
Enter shipping method (Express/Standard): Standard
Is the customer a club member? (true/false): true
Total Cost: 61.46

Purchase #2
Total Cost: 70.16
Enter shipping method (Express/Standard): Express
Is the customer a club member? (true/false): false
Total Cost: 80.16

Purchase #3
Total Cost: 41.84
Enter shipping method (Express/Standard): Standard
Is the customer a club member? (true/false): false
Total Cost: 46.84

Purchase #4
Total Cost: 87.64
Enter shipping method (Express/Standard): Standard
Is the customer a club member? (true/false): true
Total Cost: 90.64

Purchase #5
Total Cost: 55.7
Enter shipping method (Express/Standard): Express
Is the customer a club member? (true/false): true
Total Cost: 63.7
```

כתיבה ושימוש בפעולה :

הזמנת מוצרים

סוגי פעולות : פעולות המחזירות ערך

```
static void PrintAvg(int n1, int n2)
{
    double avg = (n1 + n2) / 2.0;

    Console.WriteLine(avg);
}
```

- עד כה כתבנו פעולות **void**, ללא החזרת ערך.
- לעיתים יש לנו צורך בקבלת התשובה, תוצאת חישוב, מהפעולה.
- הפעולה PrintAvg **מחשבת ממוצע ומדפיסה אותו**.
- לא ניתן להשתמש בה לצורך מציאת הממוצע הגבוה ביותר מבין כל הממוצעים של כלל תלמידי הכיתה.

סוגי פעולות : פעולות המחזרות ערך

```
static void PrintAvg (int n1, int n2)
{
    double avg = (n1 + n2) / 2.0;

    Console.WriteLine(avg);
}

static double Avg (int n1, int n2)
{
    double avg = (n1 + n2) / 2.0;

    return avg;
}
```

- הפעולה Avg מחשבת את הממוצע ומחזירה אותו.

- כאשר הפעולה מחזירה ערך, ניתן להשתמש בערך שהיא מחזירה לצורך חישובים אחרים:

```
static void Main(string[] args)
{
    int grade1, grade2;
    double avg;

    grade1 = int.Parse(Console.ReadLine());
    grade2 = int.Parse(Console.ReadLine());
    avg = Avg (grade1, grade2);
    if (avg > 90)
        Console.WriteLine("Well Done");
}
```

סוגי פעולות : פעולות המחזירות ערך

```
static void PrintAvg (int n1, int n2)
{
    double avg = (n1 + n2) / 2.0;

    Console.WriteLine(avg);
}

static double Avg (int n1, int n2)
{
    double avg = (n1 + n2) / 2.0;

    return avg;
}
```

- בפעולה **המחזירה ערך**, יש לכתוב את **טיפוס הערך** המוחזר בכותרת הפעולה.
- הפעולה **חייבת** להחזיר ערך מהטיפוס המצויין, באמצעות הפקודה **.return**

סוגי פעולות : פעולות המחזירות ערך

```
static int Max (int n1, int n2, int n3)
{
    if (n1 > n2)
    {
        if (n1 > n3)
            return n1;
        else
            return n3;
    }
    else
    {
        if (n2 > n3)
            return n2;
        else
            return n3;
    }
}
```

- בפעולה שמחזירה ערך,
 - חייבת להיות לפחות הוראת **return** אחת,
 - ויכול להיות שתהיינה כמה.

סוגי פעולות : פעולות המחזירות ערך

```
static int Max (int n1, int n2, int n3)
{
    if (n1 > n2)
    {
        if (n1 > n3)
            return n1;
        else
            return n3;
    }
    else
    {
        if (n2 > n3)
            return n2;
        else
            return n3;
    }
}
```

- בפעולה שמחזירה ערך,
 - חייבת להיות לפחות הוראת **return** אחת,
 - ויכול להיות שתהיינה כמה.
- ביצוע הוראת **return**,
 - מביא לסיום הפעולה,
 - גם אם קיימות הוראות נוספות אחריה.

סוגי פעולות : פעולות המחזירות ערך

- הזימון **שולח ערך**, ולכן אין חשיבות לשם המשתנה.
- הפעולה **מחזירה ערך**, ולכן אין חשיבות לשם המשתנה.

```
static void Main(string[] args)
{
    int x1, x2, x3, maxNum;
    Random rnd = new Random();
    for (int i = 0; i < 10; i++)
    {
        x1 = rnd.Next(101);
        x2 = rnd.Next(101);
        x3 = rnd.Next(101);
        maxNum = Max (x1, x2, x3);
        Console.WriteLine("\n Max of " + x1 + ", " + x2 + ", " + x3 + " = " + maxNum);
    }
}
```

```
static int Max (int n1, int n2, int n3)
{
    if (n1 > n2)
    {
        if (n1 > n3)
            return n1;
        else
            return n3;
    }
    else
    {
        if (n2 > n3)
            return n2;
        else
            return n3;
    }
}
```

סוגי פעולות : פעולות המחזירות ערך

```
static void Main(string[] args)
{
    int x1, x2, x3, maxNum;
    Random rnd = new Random();
    for (int i = 0; i < 10; i++)
    {
        x1 = rnd.Next(101);
        x2 = rnd.Next(101);
        x3 = rnd.Next(101);
        maxNum = Max(x1, x2, x3);
        Console.WriteLine("\n Max of " + x1 + ", " + x2 + ", " + x3 + " = " + maxNum);
    }
}
```

Max of 18, 20, 52 = 52

Max of 22, 8, 63 = 63

Max of 4, 86, 8 = 86

Max of 25, 19, 49 = 49

Max of 7, 29, 60 = 60

Max of 10, 4, 62 = 62

Max of 96, 39, 48 = 96

Max of 3, 37, 52 = 52

Max of 89, 46, 20 = 89

Max of 11, 96, 88 = 96

סוגי פעולות : פעולות המחזירות ערך

```
static int Max(int n1, int n2, int n3)
{
    if (n1 > n2)
    {
        if (n1 > n3)
            return n1;
        else
            return n3;
    }
    else
    {
        if (n2 > n3)
            return n2;
        else
            return n3;
    }
}
```

- בפעולה שמחזירה ערך,
 - חייבת להיות לפחות הוראת **return** אחת,
 - ויכול להיות שתהיינה כמה.
- ביצוע הוראת **return**,
 - מביא לסיום הפעולה,
 - גם אם קיימות הוראות נוספות אחריה.
- מאחר וכך, ניתן לוותר על הוראות ה-**else**.

סוגי פעולות : פעולות המחזירות ערך

```
static int Max(int n1, int n2, int n3)
{
    if (n1 > n2)
    {
        if (n1 > n3)
            return n1;
        return n3;
    }
    if (n2 > n3)
        return n2;
    return n3;
}
```

- בפעולה שמחזירה ערך,
 - חייבת להיות לפחות הוראת **return** אחת,
 - ויכול להיות שתהיינה כמה.
- ביצוע הוראת **return**,
 - מביא לסיום הפעולה,
 - גם אם קיימות הוראות נוספות אחריה.
- מאחר וכך, ניתן לוותר על הוראות ה-**else**.

סוגי פעולות : פעולות המחזירות ערך

```
static int Max(int n1, int n2, int n3)
{
    if (n1 > n2)
    {
        if (n1 > n3)
            return n1;
        return n3;
    }
    if (n2 > n3)
        return n2;
    return n3;
}
```

אם $n1 > n2 \ \&\& \ n1 > n3$
- יוחזר $n1$ והפעולה תסתיים.
- יתר ההוראות לא תתבצענה.

אם הגענו לכאן, $n3 \geq n1 > n2$
- יוחזר $n3$ והפעולה תסתיים.
- יתר ההוראות לא תתבצענה.

אם הגענו לכאן, $n2 \geq n1 > n3$
- יוחזר $n2$ והפעולה תסתיים.
- יתר ההוראות לא תתבצענה.

אם הגענו לכאן, $n2 \geq n1$

אם הגענו לכאן, $n3 \geq n2 \geq n1$

בפעולה המחזירה ערך,
כל חלקי הקוד חייבים להחזיר ערך !

```
static int Max(int x, int y)
{
    if (x > y)
        return x;
    return y;
}
```

שימוש בערך המוחזר מהפעולה

- ניתן להשתמש בערך המוחזר כבכל ערך מטיפוס זה.
- דוגמאות:

```
static void Main(string[] args)
{
```

```
    int num1, num2, max;
```

```
    Console.WriteLine("Enter 2 numbers: ");
```

```
    num1 = int.Parse(Console.ReadLine());
```

```
    num2 = int.Parse(Console.ReadLine());
```

```
    max = Max(num1, num2);
```

```
    Console.WriteLine("The maximum is " + max);
```

```
    Console.WriteLine("The maximum is " + Max(num1, num2));
```

```
    string s = "second";
```

```
    if (Max(num1, num2) == num1) s = "first";
```

```
    Console.WriteLine("The maximum is your " + s + " number");
```

```
}
```

1. השמה למשתנה

2. הדפסה

3. שימוש בתנאי

סוגי פעולות : פעולות המחזירות ערך

- כאשר פעולה מחזירה ערך – ניתן להשתמש בו בכל דרך בה משתמשים בערך מטיפוס הערך המוחזר.

- `int max = Max(x1, x2, x3);` // השמה למשתנה
- `if (Max(x1, x2, x3) > 7) { ... }` // שילוב בהוראת תנאי
- `Console.WriteLine(Max(x1, x2, x3));` // קלט לפעולה אחרת
- `double n = Math.Sqrt(Max(x1, x2, x3));` // קלט לפעולה אחרת
- `int max2 = Max(Max(x1, x2, x3), 10, 2);` // קלט לפעולה אחרת
- `Max(x1, x2, x3);`

אם נזמן את הפעולה, אך לא נשתמש בערך המוחזר, עדיין הפעולה תתבצע !

סוגי פעולות : פעולות המחזירות ערך

```
static double Sqrt(int num)
static int Power(int a, int b)
```

```
static void DrawSquare(int size)
```

- הערך המוחזר יכול להיות כל אחד מהטיפוסים שאנחנו מכירים
החזרה ← ניתן לאכסן ערך במשתנה כלשהו
`result = Power(basis, exponent);`
שימו לב: הדפסה למסך אינה החזרה של ערך!
- אם הפעולה אינה מחזירה ערך, נרשום `void` כערך המוחזר.

כתיבת פעולה : סיכום חלקי



- **שם הפעולה** - שם משמעותי, כתוב באותיות גמל. ✓
מילים מחוברות המתחילות באות גדולה.
- הפעולות תהיינה **static** (בשלב זה).
- **הוראות לביצוע** של הפעולה – הוראות רגילות בשפה, המשתנים המוגדרים בפעולה וכן הפרמטרים שהפעולה מקבלת הם **משתנים מקומיים**.
- אם הפעולה **מחזירה ערך**, יצוין **טיפוס הערך** בכותרת הפעולה, אחרת יכתב **void**.
- בפעולות המחזירות ערך :
 - ✓ חייבת להיות **הוראת return** אחת לפחות.
 - ✓ חייבת להיות **התאמה** בין הטיפוס המוחזר בכותרת לערך המוחזר בהוראות ה-**return**.
 - ✓ הוראת ה-**return** מסיימת את ביצוע הפעולה - ההוראות אחריה לא מתבצעות.

כתיבת פעולה : תיעוד

- יש לתעד כל פעולה
(אם בהערת שורה // או בהערת קטע /* */) •

- התיעוד כולל טענת כניסה וטענת יציאה. •

- טענת כניסה : •

- ✓ מהם הפרמטרים שהפעולה מקבלת.
- ✓ הנחות, אם יש, על הפרמטרים.

- טענת יציאה : •

- ✓ פעולה המחזירה ערך – מה הפעולה מחזירה.
- ✓ פעולת void – מה הפעולה מבצעת.

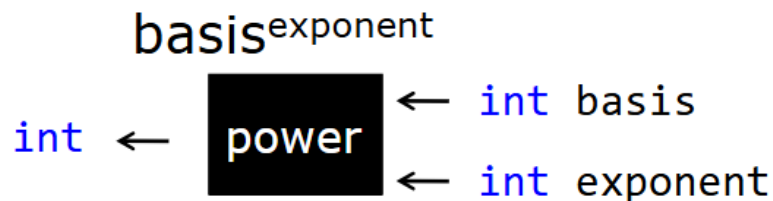
```
/*
 * הפעולה מקבלת מספר שלם
 * הפעולה מחזירה את מספר הספרות של המספר
 */
static int CountDigits (int num) ...

// הפעולה מקבלת מספר תלמידים
// הפעולה קולטת את ציוני התלמידים
// ומחזירה את הציון הממוצע
static double AvgGrade (int students) ...

/*
 * טענת כניסה: הפעולה מקבלת מחרוזת
 * טענת יציאה: הפעולה מחזירה אמת אם המחרוזת היא פלינדרום
 * אחרת – מחזירה שקר
 */
static bool IsPalindrome (string str) ...

// טענת כניסה: הפעולה מקבלת מספר שורות מבוקש ומספר עמודה מבוקש
// טענת יציאה: הפעולה מדפיסה את לוח הכפל בגודל זה
static void PrintMultiplicationTable (int row, int col) ...
```


כתיבת פעולה : שילוב בקוד



```
static int Power(int basis, int exponent)
{
    int result = 1;

    for (int i = 0; i < exponent; i++)
        result *= basis;

    return result;
}

static void Main(string[] args)
{
    int result = Power(2, 3);
    Console.WriteLine(basis + "^" + exponent + " = " + result);
}
```

1. כותרת הפעולה

- ✓ מה הפעולה עושה ?
- ✓ מה הפעולה מקבלת ?
- ✓ מה הפעולה מחזירה ?

2. מימוש

3. שימוש (זימון)

פעולה המחזירה ערך : שילוב בקוד

```
static int Power(int a, int b)
{
    int result = 1;

    for (int i = 0; i < b; i++)
        result *= a;

    return result;
}
```

טיפוס המידע
שהפעולה מחזירה

הערך שהפעולה מחזירה

```
static void Main(string[] args)
{
    int basis, exponent, result;

    Console.Write("Enter basis: ");
    basis = int.Parse(Console.ReadLine());

    Console.Write("Enter exponent: ");
    exponent = int.Parse(Console.ReadLine());

    result = Power(basis, exponent);
    Console.WriteLine(basis + "^" + exponent + " = " + result);
}
```

איחסון הערך שהפעולה מחזירה

Enter basis: 2
Enter exponent: 3
 $2^3 = 8$

Enter basis: 3
Enter exponent: 2
 $3^2 = 9$

פעולה המחזירה ערך : שילוב בקוד

```
static int Power(int a, int b)
{
    int result = 1;

    for (int i = 0; i < b; i++)
        result *= a;

    return result;
}
```

```
static void Main(string[] args)
{
    int basis, exponent, result;

    Console.Write("Enter basis: ");
    basis = int.Parse(Console.ReadLine());

    Console.Write("Enter exponent: ");
    exponent = int.Parse(Console.ReadLine());

    result = Power(basis, exponent);
    Console.WriteLine(basis + "^" + exponent + " = " + result);
}
```

```
Enter basis: 2
Enter exponent: 3
2^3 = 8
```

```
Enter basis: 3
Enter exponent: 2
3^2 = 9
```

שם המשתנה המוחזר מהפעולה
ושם המשתנה השומר את התוצאה החוזרת ב-Main
לא חייבים להיות זהים

פעולה המחזירה ערך :

שילוב בקוד

```
static int Power(int a, int b)
{
    int result = 1;

    for (int i = 0; i < b; i++)
        result *= a;

    return result;
}
```

```
static void Main(string[] args)
{
    int basis, exponent, result;

    Console.Write("Enter basis: ");
    basis = int.Parse(Console.ReadLine());

    Console.Write("Enter exponent: ");
    exponent = int.Parse(Console.ReadLine());

    result = Power(basis, exponent);
    Console.WriteLine(basis + "^" + exponent + " = " + result);
}
```

```
Enter basis: 2
Enter exponent: 3
2^3 = 8
```

```
Enter basis: 3
Enter exponent: 2
3^2 = 9
```

נשים לב כי בזימון הפעולה מעבירים רק את שמות המשתנים, ולא את הטיפוסים, בניגוד לכותרת הפעולה !

הזיכרון

int: basis	2
int: exponent	3
int: result	???

הזיכרון של ה-Main

Main

מחסנית הקריאות

```
static int Power(int a, int b)
{
    int result = 1;

    for (int i = 0; i < b; i++)
        result *= a;

    return result;
}

static void Main(string[] args)
{
    int basis, exponent, result;

    Console.Write("Enter basis: ");
    basis = int.Parse(Console.ReadLine());

    Console.Write("Enter exponent: ");
    exponent = int.Parse(Console.ReadLine());

    result = Power(basis, exponent);
    Console.WriteLine(basis + "^" + exponent + " = " + result);
}
```

```
static int Power(int a, int b)
{
    int result = 1;

    for (int i = 0; i < b; i++)
        result *= a;

    return result;
}
```

```
static void Main(string[] args)
{
    int basis, exponent, result;

    Console.Write("Enter basis: ");
    basis = int.Parse(Console.ReadLine());

    Console.Write("Enter exponent: ");
    exponent = int.Parse(Console.ReadLine());

    result = Power(basis, exponent);
    Console.WriteLine(basis + "^" + exponent + " = " + result);
}
```

הזיכרון

int: a	2
int: b	3
int: i	???
int: result	???

הזיכרון של הפעולה power

int: basis	2
int: exponent	3
int: result	???

הזיכרון של ה-Main

Power (2,3)
Main (line 6)

מחסנית הקריאות

הזיכרון

```
static int Power(int a, int b)
{
    int result = 1;

    for (int i = 0; i < b; i++)
        result *= a;

    return result;
}
```

```
static void Main(string[] args)
{
    int basis, exponent, result;
```

```
    Console.Write("Enter basis: ");
    basis = int.Parse(Console.ReadLine());
```

```
    Console.Write("Enter exponent: ");
    exponent = int.Parse(Console.ReadLine());
```

```
    result = Power(basis, exponent);
    Console.WriteLine(basis + "^" + exponent + " = " + result);
}
```

int: a	2
int: b	3
int: i	...
int: result	8

הזיכרון של הפעולה power

int: basis	2
int: exponent	3
int: result	???

הזיכרון של ה-Main

Power (2,3)
Main (line 6)

מחסנית הקריאות

```
static int Power(int a, int b)
{
    int result = 1;

    for (int i = 0; i < b; i++)
        result *= a;

    return result;
}
```

```
static void Main(string[] args)
{
    int basis, exponent, result;

    Console.Write("Enter basis: ");
    basis = int.Parse(Console.ReadLine());

    Console.Write("Enter exponent: ");
    exponent = int.Parse(Console.ReadLine());

    result = Power(basis, exponent);
    Console.WriteLine(basis + "^" + exponent + " = " + result);
}
```

הזיכרון

int: basis	2
int: exponent	3
int: result	8

הזיכרון של ה-Main

Main
(line 6)

מחסנית הקריאות


```
static int Power(int a, int b)
{
    int result = 1;

    for (int i = 0; i < b; i++)
        result *= a;

    return result;
}
```

```
static void Main(string[] args)
{
    int basis, exponent, res;
```

```
    Console.Write("Enter basis: ");
    basis = int.Parse(Console.ReadLine());
```

```
    Console.Write("Enter exponent: ");
    exponent = int.Parse(Console.ReadLine());
```

```
    res = Power(basis, exponent);
    Console.WriteLine(basis + "^" + exponent + " = " + result);
```

```
}
```

המשתנה result שב-Main
זוהי שבפעולה אינם אותו משתנה,
ולכן אין הכרח שהם יהיו עם שם זהה !

כיצד נראה הזיכרון : סיכום

- לכל פעולה יש שטח נפרד בו מוגדרים המשתנים שלה.
- פעולה יכולה לגשת לשטח הזיכרון שלה בלבד, לכן אינה יכולה לגשת למשתנים שהוגדרו בפעולות אחרות.
- ✓ הפרמטרים המועברים לפעולה הם גם משתנים של הפעולה.
- ✓ הפרמטרים המועברים לפעולה הם העתקים של ערכי המשתנים שנשלחו, ולכן העברת פרמטרים כזו נקראת **by value**.
- ✓ כל שינוי בהם הוא מקומי לפעולה, ולא משנה את ערכי המשתנים המקוריים (שנשלחו).
- כאשר יוצאים מפעולה, התאים במחסנית שבהם נשמרו ערכיה נמחקים, כל קריאה לפעולה תייצר אותם מחדש.



כתיבה ושימוש בפעולה: האם פלינדרום?

פלינדרום היא מילה סימטרית, כמו למשל: אלמלא, מהמהם, תוססות.

מספרים שהם פלינדרום: 91819, 437734.

כתבו פעולה המקבלת מספר טבעי ומחזירה אמת אם המספר הוא פלינדרום, אחרת מחזירה שקר.

```
// הפעולה מקבלת מספר טבעי
// הפעולה מחזירה אמת אם המספר הוא פלינדרום, שקר אחרת
static bool IsPalindrome(int num)
{
    int digit, newNum = 0, n = num;
    while (n > 0)
    {
        digit = n % 10;
        newNum = newNum * 10 + digit;
        n /= 10;
    }
    if (newNum == num)
        return true;
    return false;
}
```

return newNum == num;

ביטויים אלו שקולים.
אחרי הוראת ה-return יכול לבוא כל ביטוי
שהערך שלו הוא מטיפוס הערך המוחזר.
קודם יחושב ערך הביטוי,
ואח"כ יוחזר הערך שלו ע"י הוראת ה-return.

כתיבה ושימוש בפעולה : האם פלינדרום ?

כתבו קטע קוד המגריל ומדפיס מספרים רנדומליים בני 4-5 ספרות עד אשר יוגרל מספר פלינדרומי.
יש להשתמש בפעולה `.IsPalindrome`

העצם מטיפוס `Random` יוגדר
תמיד פעם אחד בלבד !

```
Random rnd = new Random();  
int n = rnd.Next(1000, 100000);  
Console.WriteLine(n);  
  
while(!IsPalindrome(n)) // same as: IsPalindrome(n) == false  
{  
    n = rnd.Next(1000, 100000);  
    Console.WriteLine(n);  
}
```

כתיבה ושימוש בפעולה : תיאור מזג האוויר היום

יום חמים (warm) הוא יום שהטמפרטורה בו היא בין 20.0° (מעלות צלסיוס) ל- 30.0° , אם הטמפרטורה גבוהה יותר מדובר ביום חם (hot), ואם נמוכה מכך, מדובר ביום קר (cool).
יום יכול להיות גשום (rainy) או לא.

כתבו פעולה המקבלת את הטמפרטורה שנמדדה ומשתנה בוליאני שאם ערכו אמת, ירד גשם ביום זה.
הפעולה תחזיר מחרוזת המתארת את מזג האוויר באותו יום.

לדוגמה, אם הטמפרטורה הייתה 40.0° וירד גשם, הפעולה תחזיר את המחרוזת : **It's hot and rainy.**

כתיבה ושימוש בפעולה : תיאור מזג האוויר היום

יום חמים (warm) הוא יום שהטמפרטורה בו היא בין 20.0° (מעלות צלסיוס) ל- 30.0° , אם הטמפרטורה גבוהה יותר מדובר ביום חם (hot), ואם נמוכה מכך, מדובר ביום קר (cool).
יום יכול להיות גשום (rainy) או לא.

כתבו פעולה המקבלת את הטמפרטורה שנמדדה ומשתנה בוליאני שאם ערכו אמת, ירד גשם ביום זה.
הפעולה תחזיר מחרוזת המתארת את מזג האוויר באותו יום.

לדוגמה, אם הטמפרטורה הייתה 40.0° וירד גשם, הפעולה תחזיר את המחרוזת : **It's hot and rainy.**

```
static string DetermineWeatherCondition(double temperatureCelsius, bool isRaining)
{
    string str = "It's ";
    if (temperatureCelsius > 30.0)
        str = "hot";
    else if (temperatureCelsius >= 20.0)
        str = "warm";
    else
        str = "cool";
    str += " and ";
    if (isRaining) // same as isRaining == true
        return str + "rainy.";
    return str + "sunny.";
}
```

// הפעולה מקבלת טמפרטורה
// ומשתנה בוליאני שאם ירד גשם ערכו אמת, אחרת שקר
// הפעולה מחזירה מחרוזת המתארת את מזג האוויר

נשתדל שתהיה פקודת return אחת
בסיום הפעולה.

כתיבה ושימוש בפעולה : תיאור מזג האוויר היום

יום חמים (warm) הוא יום שהטמפרטורה בו היא בין 20.0° (מעלות צלסיוס) ל- 30.0° , אם הטמפרטורה גבוהה יותר מדובר ביום חם (hot), ואם נמוכה מכך, מדובר ביום קר (cool).
יום יכול להיות גשום (rainy) או לא.

כתבו פעולה המקבלת את הטמפרטורה שנמדדה ומשתנה בוליאני שאם ערכו אמת, ירד גשם ביום זה.
הפעולה תחזיר מחרוזת המתארת את מזג האוויר באותו יום.

לדוגמה, אם הטמפרטורה הייתה 40.0° וירד גשם, הפעולה תחזיר את המחרוזת: **It's hot and rainy.**

```
static string DetermineWeatherCondition(double temperatureCelsius, bool isRaining)
{
    string str = "It's ";
    if (temperatureCelsius > 30.0)
        str = "hot";
    else if (temperatureCelsius >= 20.0)
        str = "warm";
    else
        str = "cool";
    str += " and ";
    if (isRaining) // same as isRaining == true
        str += "rainy.";
    else str += "sunny.";
    return str;
}
```

// הפעולה מקבלת טמפרטורה ומשתנה בוליאני שאם ירד גשם ערכו אמת, אחרת שקר
// הפעולה מחזירה מחרוזת המתארת את מזג האוויר

נשתדל שתהיה פקודת **return** אחת
בסיום הפעולה.

כתיבה ושימוש בפעולה :

תיאור מזג האוויר היום

כתבו קטע קוד הקולט את הטמפרטורה שירדה בכל יום מימי השבוע, והאם ירד גשם באותו יום.

הקטע ידפיס את תיאור מזג האוויר לכל יום.

יש להשתמש בפעולה `DetermineWeatherCondition`

```
double temp;
bool rain;
for (int i = 0; i < 7; i++)
{
    Console.Write("temperature (Celsius)): ");
    temp = double.Parse(Console.ReadLine());
    Console.Write("raining? (true/false)): ");
    rain = bool.Parse(Console.ReadLine());
    Console.WriteLine(DetermineWeatherCondition(temp, rain));
    Console.WriteLine();
}
```

```
temperature (Celsius)): 23.4
raining? (true/false)): true
It's warm and rainy.
```

```
temperature (Celsius)): 16.5
raining? (true/false)): false
It's cool and sunny.
```

```
temperature (Celsius)): 34.7
raining? (true/false)): true
It's hot and rainy.
```

```
temperature (Celsius)): 35.9
raining? (true/false)): false
It's hot and sunny.
```

```
temperature (Celsius)): 23.3
raining? (true/false)): true
It's warm and rainy.
```

```
temperature (Celsius)): 10.9
raining? (true/false)): true
It's cool and rainy.
```

```
temperature (Celsius)): 28.3
raining? (true/false)): false
It's warm and sunny.
```


העמסת פעולות

```
static int Min(int x, int y)
{
    if (x < y)
        return x;
    return y;
}
```

```
static int Min(int x, int y, int z)
{
    if (x < y && x < z)
        return x;
    else if (y < z)
        return y;
    else
        return z;
}
```

```
static double Min(double x, double y)
{
    if (x < y)
        return x;
    return y;
}
```

```
static void Main(string[] args)
{
    Console.WriteLine(Min(1, 2));
    Console.WriteLine(Min(4, 3, 2));
    Console.WriteLine(Min(1.3, 2.7));
}
```

1

2

1.3

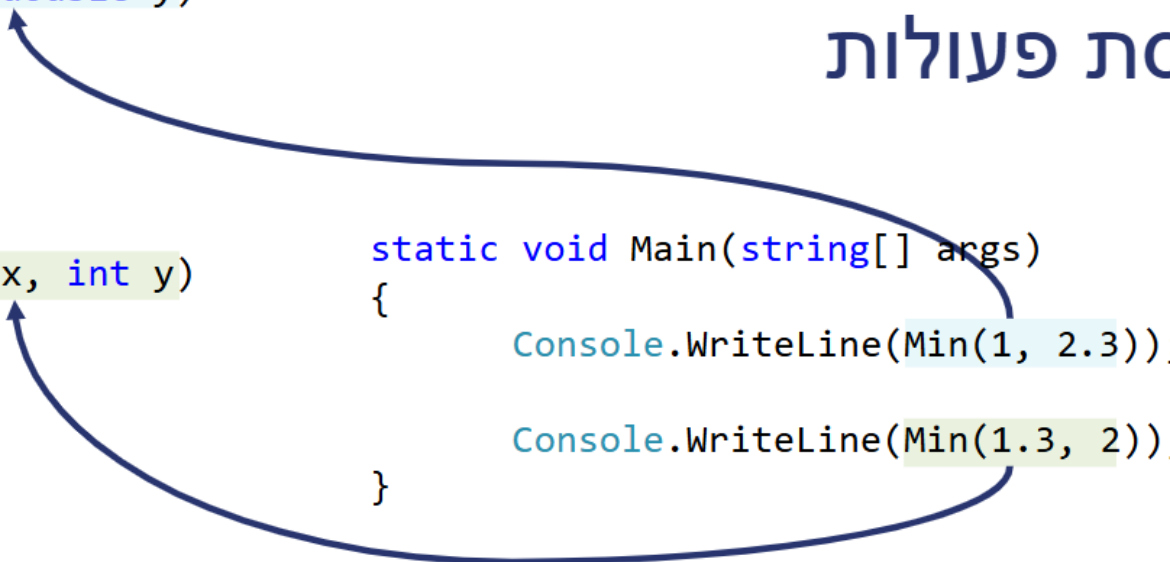
זיהוי הפעולה מתבצע
בהתאם למספר ולטיפוס הפרמטרים הנשלחים.
בהתאם לחתימה של הפעולה.

העמסת פעולות

```
static double Min(int x, double y)
{
    if (x < y) return x;
    return y;
}
```

```
static double Min(double x, int y)
{
    if (x < y) return x;
    return y;
}
```

```
static void Main(string[] args)
{
    Console.WriteLine(Min(1, 2.3));
    Console.WriteLine(Min(1.3, 2));
}
```



לפעולות אלו יש חתימה שונה,
כיוון שסדר הפרמטרים שלהם שונה.

העמסת פעולות

```
static int Min(int x, int y)
{
    if (x < y) return x;
    return y;
}
```

```
static int Min(int a, int b)
{
    if (x < y) return x;
    return y;
}
```



```
static void Main(string[] args)
{
    Console.WriteLine(Min(1, 2));
}
```



שתי הפעולות מקבלות שני מספרים שלמים

העמסת פעולות

```
static int Min(int x, int y)
{
    if (x < y) return x;
    return y;
}
```

```
static int Min(int a, int b)
{
    if (x < y) return x;
    return y;
}
```



```
static void Main(string[] args)
{
    Console.WriteLine(Min(1, 2));
}
```



שתי הפעולות מקבלות שני מספרים שלמים
לא ניתן לכתוב פעולות עם אותה חתימה !
(אותו טיפוס של פרמטרים ובאותו הסדר)

פעולות : יתרונות



- **המטרה – מודולריות**
חלוקת התוכנית לקטעי קוד יותר קטנים, המבצעים משימות משנה ספציפיות.
- **יתרונות:**
 - ✓ **קריאות** - קוד מסודר וקריא.
 - ✓ **תחזוקה** - קוד קל לבדיקה, וקל לעדכון.
תיקון/שינוי/עדכון של פעולה אחת, אינו משפיע על פעולה אחרת.
 - ✓ **שימוש חוזר בקוד** - ניתן לקרוא לפעולה ממקומות שונים בקוד,
ובעזרת לשלוח נתונים שונים לפי הצורך.
 - ✓ **חלוקת עבודה** בין מתכנתים שונים – כל אחד אחראי לחלק מהפעולות.

תכנון TOP-DOWN

- כאשר ניגשים לכתוב תוכנית, יש לחשוב מהי הבעיה הגדולה שעלינו לפתור ולחלק אותה לחלקים.
- לכל חלק "קשה" נתייחס כאל "קופסא שחורה" (שתמומש בהמשך).
- כאשר נממש כל חלק "קשה" אנו מתמודדים עם בעיה יותר קטנה שגם אותה ננסה לפרק לבעיות יותר קטנות, עד אשר נגדיר בעיות מספיק קטנות.



תכנות TOP-DOWN :

דוגמת ציור משולש

```
static void PrintLine(int length, char ch) {  
    for ( int i=1 ; i <= length ; i++ )  
        Console.Write(ch);  
    Console.WriteLine();  
}  
  
static void PrintTriangle(int basis, char ch) {  
    for( int i=1 ; i <= basis ; i++ )  
        PrintLine(i, ch);  
}  
  
static void Main(string[] args)  
{  
    int basis;  
    char ch;  
  
    Console.WriteLine("Enter basis and char: ");  
    basis = int.Parse(Console.ReadLine());  
    ch = char.Parse(Console.ReadLine());  
    PrintTriangle(basis, ch);  
}
```

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

int: basis	???
char: ch	???

הזיכרון של ה-Main

תכנות TOP-DOWN :

דוגמת ציור משולש

```
static void PrintLine(int length, char ch) {  
    for ( int i=1 ; i <= length ; i++ )  
        Console.Write(ch);  
    Console.WriteLine();  
}  
  
static void PrintTriangle(int basis, char ch) {  
    for( int i=1 ; i <= basis ; i++ )  
        PrintLine(i, ch);  
}  
  
static void Main(string[] args)  
{  
    int basis;  
    char ch;  
  
    Console.WriteLine("Enter basis and char: ");  
    basis = int.Parse(Console.ReadLine());  
    ch = char.Parse(Console.ReadLine());  
    PrintTriangle(basis, ch);  
}
```

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

תכנות TOP-DOWN :

דוגמת ציור משולש

```
static void PrintLine(int length, char ch) {  
    for ( int i=1 ; i <= length ; i++ )  
        Console.Write(ch);  
    Console.WriteLine();  
}  
  
static void PrintTriangle(int basis, char ch) {  
    for( int i=1 ; i <= basis ; i++ )  
        PrintLine(i, ch);  
}  
  
static void Main(string[] args)  
{  
    int basis;  
    char ch;  
  
    Console.WriteLine("Enter basis and char: ");  
    basis = int.Parse(Console.ReadLine());  
    ch = char.Parse(Console.ReadLine());  
    PrintTriangle(basis, ch);  
}
```

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

int: basis	3
char: ch	'\$'
int: i	1

הזיכרון של

PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

```

static void PrintLine(int length, char ch) {
    for ( int i=1 ; i <= length ; i++ )
        Console.Write(ch);
    Console.WriteLine();
}

static void PrintTriangle(int basis, char ch) {
    for( int i=1 ; i <= basis ; i++ )
        PrintLine(i, ch);
}

static void Main(string[] args)
{
    int basis;
    char ch;

    Console.WriteLine("Enter basis and char: ");
    basis = int.Parse(Console.ReadLine());
    ch = char.Parse(Console.ReadLine());
    PrintTriangle(basis, ch);
}

```

: TOP-DOWN תכנות

דוגמת ציור משולש

```

Enter basis and char:
3
$
$
$$
$$$

```

\$

int: length	1
char: ch	'\$'
int: i	2

הזיכרון של PrintLine

int: basis	3
char: ch	'\$'
int: i	1

הזיכרון של

PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

```

static void PrintLine(int length, char ch) {
    for ( int i=1 ; i <= length ; i++ )
        Console.Write(ch);
    Console.WriteLine();
}

static void PrintTriangle(int basis, char ch) {
    for( int i=1 ; i <= basis ; i++ )
        PrintLine(i, ch);
}

static void Main(string[] args)
{
    int basis;
    char ch;

    Console.WriteLine("Enter basis and char: ");
    basis = int.Parse(Console.ReadLine());
    ch = char.Parse(Console.ReadLine());
    PrintTriangle(basis, ch);
}

```

תכנות TOP-DOWN :

דוגמת ציור משולש

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$

int: length	2
char: ch	'\$'
int: i	1

הזיכרון של PrintLine

int: basis	3
char: ch	'\$'
int: i	2

הזיכרון של

PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

```

static void PrintLine(int length, char ch) {
    for ( int i=1 ; i <= length ; i++ )
        Console.Write(ch);
    Console.WriteLine();
}

static void PrintTriangle(int basis, char ch) {
    for( int i=1 ; i <= basis ; i++ )
        PrintLine(i, ch);
}

static void Main(string[] args)
{
    int basis;
    char ch;

    Console.WriteLine("Enter basis and char: ");
    basis = int.Parse(Console.ReadLine());
    ch = char.Parse(Console.ReadLine());
    PrintTriangle(basis, ch);
}

```

תכנות TOP-DOWN : דוגמת ציור משולש

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$
\$

int: length	2
char: ch	'\$'
int: i	2

הזיכרון של PrintLine

int: basis	3
char: ch	'\$'
int: i	2

הזיכרון של

PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

```

static void PrintLine(int length, char ch) {
    for ( int i=1 ; i <= length ; i++ )
        Console.Write(ch);
    Console.WriteLine();
}

static void PrintTriangle(int basis, char ch) {
    for( int i=1 ; i <= basis ; i++ )
        PrintLine(i, ch);
}

static void Main(string[] args)
{
    int basis;
    char ch;

    Console.WriteLine("Enter basis and char: ");
    basis = int.Parse(Console.ReadLine());
    ch = char.Parse(Console.ReadLine());
    PrintTriangle(basis, ch);
}

```

: TOP-DOWN תכנות

דוגמת ציור משולש

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$
\$ \$

int: length	2
char: ch	'\$'
int: i	3

הזיכרון של PrintLine

int: basis	3
char: ch	'\$'
int: i	2

הזיכרון של

PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של Main-

תכנות TOP-DOWN :

דוגמת ציור משולש

```
static void PrintLine(int length, char ch) {  
    for ( int i=1 ; i <= length ; i++ )  
        Console.Write(ch);  
    Console.WriteLine();  
}  
  
static void PrintTriangle(int basis, char ch) {  
    for( int i=1 ; i <= basis ; i++ )  
        PrintLine(i, ch);  
}  
  
static void Main(string[] args)  
{  
    int basis;  
    char ch;  
  
    Console.WriteLine("Enter basis and char: ");  
    basis = int.Parse(Console.ReadLine());  
    ch = char.Parse(Console.ReadLine());  
    PrintTriangle(basis, ch);  
}
```

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$
\$ \$

int: basis	3
char: ch	'\$'
int: i	3

הזיכרון של

PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

```

static void PrintLine(int length, char ch) {
    for ( int i=1 ; i <= length ; i++ )
        Console.Write(ch);
    Console.WriteLine();
}

static void PrintTriangle(int basis, char ch) {
    for( int i=1 ; i <= basis ; i++ )
        PrintLine(i, ch);
}

static void Main(string[] args)
{
    int basis;
    char ch;

    Console.WriteLine("Enter basis and char: ");
    basis = int.Parse(Console.ReadLine());
    ch = char.Parse(Console.ReadLine());
    PrintTriangle(basis, ch);
}

```

תכנות TOP-DOWN :

דוגמת ציור משולש

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$
\$ \$
\$

int: length	3
char: ch	'\$'
int: i	1

הזיכרון של PrintLine

int: basis	3
char: ch	'\$'
int: i	3

הזיכרון של

PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

```

static void PrintLine(int length, char ch) {
    for ( int i=1 ; i <= length ; i++ )
        Console.Write(ch);
    Console.WriteLine();
}

static void PrintTriangle(int basis, char ch) {
    for( int i=1 ; i <= basis ; i++ )
        PrintLine(i, ch);
}

static void Main(string[] args)
{
    int basis;
    char ch;

    Console.WriteLine("Enter basis and char: ");
    basis = int.Parse(Console.ReadLine());
    ch = char.Parse(Console.ReadLine());
    PrintTriangle(basis, ch);
}

```

: TOP-DOWN תכנות

דוגמת ציור משולש

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$
\$ \$
\$ \$

int: length	3
char: ch	'\$'
int: i	2

הזיכרון של PrintLine

int: basis	3
char: ch	'\$'
int: i	3

הזיכרון של

PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main


```

static void PrintLine(int length, char ch) {
    for ( int i=1 ; i <= length ; i++ )
        Console.Write(ch);
    Console.WriteLine();
}

static void PrintTriangle(int basis, char ch) {
    for( int i=1 ; i <= basis ; i++ )
        PrintLine(i, ch);
}

static void Main(string[] args)
{
    int basis;
    char ch;

    Console.WriteLine("Enter basis and char: ");
    basis = int.Parse(Console.ReadLine());
    ch = char.Parse(Console.ReadLine());
    PrintTriangle(basis, ch);
}

```

תכנות TOP-DOWN :

דוגמת ציור משולש

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$
\$ \$
\$ \$ \$

int: length	3
char: ch	'\$'
int: i	3

הזיכרון של PrintLine

int: basis	3
char: ch	'\$'
int: i	3

הזיכרון של

PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

```

static void PrintLine(int length, char ch) {
    for ( int i=1 ; i <= length ; i++ )
        Console.Write(ch);
    Console.WriteLine();
}

static void PrintTriangle(int basis, char ch) {
    for( int i=1 ; i <= basis ; i++ )
        PrintLine(i, ch);
}

static void Main(string[] args)
{
    int basis;
    char ch;

    Console.WriteLine("Enter basis and char: ");
    basis = int.Parse(Console.ReadLine());
    ch = char.Parse(Console.ReadLine());
    PrintTriangle(basis, ch);
}

```

: TOP-DOWN תכנות

דוגמת ציור משולש

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$
\$ \$
\$ \$ \$

int: basis	3
char: ch	'\$'
int: i	4

הזיכרון של
PrintTriangle

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

תכנות TOP-DOWN :

דוגמת ציור משולש

```
static void PrintLine(int length, char ch) {  
    for ( int i=1 ; i <= length ; i++ )  
        Console.Write(ch);  
    Console.WriteLine();  
}  
  
static void PrintTriangle(int basis, char ch) {  
    for( int i=1 ; i <= basis ; i++ )  
        PrintLine(i, ch);  
}  
  
static void Main(string[] args)  
{  
    int basis;  
    char ch;  
  
    Console.WriteLine("Enter basis and char: ");  
    basis = int.Parse(Console.ReadLine());  
    ch = char.Parse(Console.ReadLine());  
    PrintTriangle(basis, ch);  
}
```

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$
\$ \$
\$ \$ \$

int: basis	3
char: ch	'\$'

הזיכרון של ה-Main

```
static void PrintLine(int length, char ch) {
    for ( int i=1 ; i <= length ; i++ )
        Console.Write(ch);
    Console.WriteLine();
}

static void PrintTriangle(int basis, char ch) {
    for( int i=1 ; i <= basis ; i++ )
        PrintLine(i, ch);
}

static void Main(string[] args)
{
    int basis;
    char ch;

    Console.WriteLine("Enter basis and char: ");
    basis = int.Parse(Console.ReadLine());
    ch = char.Parse(Console.ReadLine());
    PrintTriangle(basis, ch);
}
```

: TOP-DOWN תכנות

דוגמת ציור משולש

Enter basis and char:

3
\$
\$
\$\$
\$\$\$

\$
\$ \$
\$ \$ \$

תכנות TOP-DOWN : חישוב מספר מופעים של ספרה במספר

- בהינתן מספר וספרה, יש לחשב ולהחזיר את מספר המופעים של הספרה במספר.
✓ עבור המספר 2422526 והספרה 2 יוחזר הערך 4.
✓ עבור המספר 993495969997 והספרה 9 יוחזר הערך 7.

- לכן כותרת הפעולה תראה כך:

```
static int CountDigitInNumber(int number, int digit)
```

מחזירה

מקבלת

מקבלת

תכנות TOP-DOWN : חישוב מספר מופעים של ספרה במספר

```
public static int CountDigitInNumber(int number, int digit)
{
    int count = 0;
    while(number>0){
        if (number%10 == digit)
            count++;
        number = number/10;
    }
    return count;
}
```

כתבו פעולה שתקבל
- מספר number
- וספרה digit.

הפעולה תחזיר את מספר הפעמים
שהספרה digit מופיעה במספר number.

```
public static void Main(String[] args) {
    int number = 2422526, digit = 2;
    Console.WriteLine( CountDigitInNumber (number,digit) );
    int a= 993495969997, b= 9;
    Console.WriteLine( CountDigitInNumber (a, b) );
}
```

4

תכנות TOP-DOWN : חישוב מספר מופעים של ספרה במספר

```
public static int CountDigitInNumber(int number, int digit)
{
    int count = 0;
    while(number>0){
        if (number%10 == digit)
            count++;
        number = number/10;
    }
    return count;
}
```

כתבו פעולה שתקבל

- מספר number
- וספרה digit.

הפעולה תחזיר את מספר הפעמים
שהספרה digit מופיעה במספר number.

```
public static void Main(String[] args) {
    int number = 2422526, digit = 2;
    Console.WriteLine( CountDigitInNumber (number,digit) );
    int a= 993495969997, b= 9;
    Console.WriteLine( CountDigitInNumber (a, b) );
}
```

4

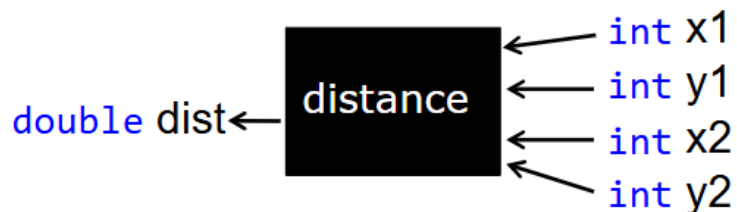
7

תכנות TOP-DOWN : דוגמת חישוב מרחק בין 2 נקודות

- בהינתן 2 הנקודות (x_1, y_1) ו- (x_2, y_2) הנוסחה לחישוב המרחק ביניהן:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- נכתוב פעולה המקבלת 4 קואורדינטות המייצגות 2 נקודות, ומחזירה את המרחק ביניהן:



- לכן כותרת הפעולה תראה כך:

```
static double Distance(int x1, int y1, int x2, int y2)
```


דוגמת חישוב מרחק בין 2 נקודות : באילו פעולות נשתמש

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- כדי לחשב מרחק בין שתי נקודות צריך לדעת לחשב חזקה ושורש, לכן נגדיר ונשתמש ב- 2 פעולות עזר:

1.Sqrt המקבלת מספר **double** ומחזירה את השורש שלו. פעולה זו כבר קיימת במחלקת **Math**.

2.Power שאותה נכתוב בעצמנו לצורך התרגול (למרות שהיא קיימת במחלקת **Math**).

```
Please enter 2 points (x1, y1, x2, y2):
```

```
0
```

```
0
```

```
4
```

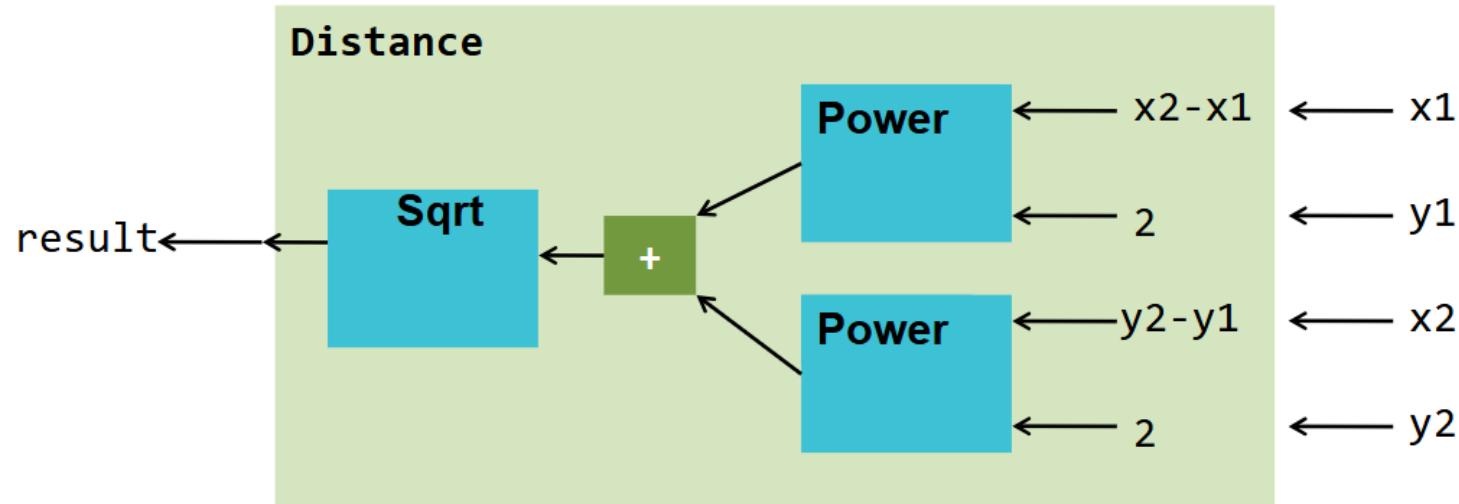
```
4
```

```
The distance 5.65685424949238
```

$$\sqrt{(4-0)^2 + (4-0)^2} = \sqrt{4^2 + 4^2} = \sqrt{32}$$

דוגמת חישוב מרחק בין 2 נקודות : תרשים הפעולה

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	???
int: y1	???
int: x2	???
int: y2	???
double: dist	???

הזיכרון של ה-Main

```
static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}
```

```
static void Main(string[] args) {
    int x1, y1, x2, y2;

    Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
    x1 = int.Parse(Console.ReadLine());
    y1 = int.Parse(Console.ReadLine());
    x2 = int.Parse(Console.ReadLine());
    y2 = int.Parse(Console.ReadLine());

    double dist = Distance(x1, y1, x2, y2);
    Console.WriteLine("The distance " + dist);
}
```

Main

מחסנית הקריאות

```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: dist	???

הזיכרון של ה-Main

```
static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}
```

```
static void Main(string[] args) {
    int x1, y1, x2, y2;

    Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
    x1 = int.Parse(Console.ReadLine());
    y1 = int.Parse(Console.ReadLine());
    x2 = int.Parse(Console.ReadLine());
    y2 = int.Parse(Console.ReadLine());

    double dist = Distance(x1, y1, x2, y2);
    Console.WriteLine("The distance " + dist);
}
```

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: xDiffPower	???
double: yDiffPower	???
double: result	???

הזיכרון של Distance

Distance
Main (line 8)

מחסנית הקריאות

```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}
```

```
static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}
```

```
static void Main(string[] args) {
    int x1, y1, x2, y2;

    Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
    x1 = int.Parse(Console.ReadLine());
    y1 = int.Parse(Console.ReadLine());
    x2 = int.Parse(Console.ReadLine());
    y2 = int.Parse(Console.ReadLine());

    double dist = Distance(x1, y1, x2, y2);
    Console.WriteLine("The distance " + dist);
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: dist	???

הזיכרון של ה-Main

int: basis	4
int: exponent	2
int: i	???
int: result	16

הזיכרון של Power

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: xDiffPower	???
double: yDiffPower	???
double: result	???

הזיכרון של Distance

Power
Distance (line 3)
Main (line 8)

מחסנית הקריאות

```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}
```

```
static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}
```

```
static void Main(string[] args) {
    int x1, y1, x2, y2;

    Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
    x1 = int.Parse(Console.ReadLine());
    y1 = int.Parse(Console.ReadLine());
    x2 = int.Parse(Console.ReadLine());
    y2 = int.Parse(Console.ReadLine());

    double dist = Distance(x1, y1, x2, y2);
    Console.WriteLine("The distance " + dist);
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: dist	???

הזיכרון של ה-Main

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: xDiffPower	16
double: yDiffPower	???
double: result	???

הזיכרון של Distance

Distance
Main (line 8)

מחסנית הקריאות

```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: dist	???

הזיכרון של ה-Main

```
static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}
```

int: basis	4
int: exponent	2
int: i	???
int: result	16

הזיכרון של Power

```
static void Main(string[] args) {
    int x1, y1, x2, y2;
```

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: xDiffPower	16
double: yDiffPower	???
double: result	???

הזיכרון של Distance

```
Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
```

```
x1 = int.Parse(Console.ReadLine());
y1 = int.Parse(Console.ReadLine());
x2 = int.Parse(Console.ReadLine());
y2 = int.Parse(Console.ReadLine());
```

```
double dist = Distance(x1, y1, x2, y2);
Console.WriteLine("The distance " + dist);
```

```
}
```

Power
Distance (line 2)
Main (line 8)

מחסנית הקריאות

```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: dist	???

הזיכרון של ה-Main

```
static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}
```

int: basis	4
int: exponent	2
int: i	???
int: result	16

הזיכרון של Power

```
static void Main(string[] args) {
    int x1, y1, x2, y2;
```

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: xDiffPower	16
double: yDiffPower	???
double: result	???

הזיכרון של Distance

```
Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
```

```
x1 = int.Parse(Console.ReadLine());
```

```
y1 = int.Parse(Console.ReadLine());
```

```
x2 = int.Parse(Console.ReadLine());
```

```
y2 = int.Parse(Console.ReadLine());
```

```
double dist = Distance(x1, y1, x2, y2);
```

```
Console.WriteLine("The distance " + dist);
```

```
}
```

Power
Distance (line 2)
Main (line 8)

מחסנית הקריאות


```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}
```

```
static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}
```

```
static void Main(string[] args) {
    int x1, y1, x2, y2;

    Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
    x1 = int.Parse(Console.ReadLine());
    y1 = int.Parse(Console.ReadLine());
    x2 = int.Parse(Console.ReadLine());
    y2 = int.Parse(Console.ReadLine());

    double dist = Distance(x1, y1, x2, y2);
    Console.WriteLine("The distance " + dist);
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: dist	???

הזיכרון של ה-Main

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: xDiffPower	16
double: yDiffPower	16
double: result	???

הזיכרון של Distance

Distance
Main (line 8)

מחסנית הקריאות

```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}
```

```
static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}
```

```
static void Main(string[] args) {
    int x1, y1, x2, y2;

    Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
    x1 = int.Parse(Console.ReadLine());
    y1 = int.Parse(Console.ReadLine());
    x2 = int.Parse(Console.ReadLine());
    y2 = int.Parse(Console.ReadLine());

    double dist = Distance(x1, y1, x2, y2);
    Console.WriteLine("The distance " + dist);
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: dist	???

הזיכרון של ה-Main

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: xDiffPower	16
double: yDiffPower	16
double: result	???

הזיכרון של Distance

Sqrt
Distance (line 3)
Main (line 8)

מחסנית הקריאות

```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: dist	???

הזיכרון של ה-Main

```
static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}
```

```
static void Main(string[] args) {
    int x1, y1, x2, y2;

    Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
    x1 = int.Parse(Console.ReadLine());
    y1 = int.Parse(Console.ReadLine());
    x2 = int.Parse(Console.ReadLine());
    y2 = int.Parse(Console.ReadLine());

    double dist = Distance(x1, y1, x2, y2);
    Console.WriteLine("The distance " + dist);
}
```

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: xDiffPower	16
double: yDiffPower	16
double: result	5.6

הזיכרון של Distance

Distance
Main
(line 8)

מחסנית הקריאות

```
static double Distance(int x1, int y1, int x2, int y2) {
```

```
    double xDiffPower, yDiffPower, result;
```

```
    xDiffPower = Power(x2-x1, 2);
```

```
    yDiffPower = Power(y2-y1, 2);
```

```
    result = Math.Sqrt(xDiffPower+yDiffPower);
```

```
    return result;
```

```
}
```

```
static int Power(int basis, int exponent) {
```

```
    int result=1;
```

```
    for (int i=0 ; i < exponent ; i++)
```

```
        result *= basis;
```

```
    return result;
```

```
}
```

```
static void Main(string[] args) {
```

```
    int x1, y1, x2, y2;
```

```
    Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
```

```
    x1 = int.Parse(Console.ReadLine());
```

```
    y1 = int.Parse(Console.ReadLine());
```

```
    x2 = int.Parse(Console.ReadLine());
```

```
    y2 = int.Parse(Console.ReadLine());
```

```
    double dist = Distance(x1, y1, x2, y2);
```

```
    Console.WriteLine("The distance " + dist);
```

```
}
```

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

int: x1	0
int: y1	0
int: x2	4
int: y2	4
double: dist	5.6

הזיכרון של ה-Main

Main

מחסנית הקריאות

חישוב מרחק בין 2 נקודות

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

```
static double Distance(int x1, int y1, int x2, int y2) {
    double xDiffPower, yDiffPower, result;

    xDiffPower = Power(x2-x1, 2);
    yDiffPower = Power(y2-y1, 2);

    result = Math.Sqrt(xDiffPower+yDiffPower);
    return result;
}

static int Power(int basis, int exponent) {
    int result=1;
    for (int i=0 ; i < exponent ; i++)
        result *= basis;
    return result;
}

static void Main(string[] args) {
    int x1, y1, x2, y2;

    Console.WriteLine("Please enter 2 points (x1, y1, x2, y2): ");
    x1 = int.Parse(Console.ReadLine());
    y1 = int.Parse(Console.ReadLine());
    x2 = int.Parse(Console.ReadLine());
    y2 = int.Parse(Console.ReadLine());

    double dist = Distance(x1, y1, x2, y2);
    Console.WriteLine("The distance " + dist);
}
```

דוגמה מסכמת : תקינות תאריך

1. ינואר: 31 ימים
2. פברואר: בשנה לא מעוברת 28 ימים
בשנה מעוברת 29 ימים
3. מרץ: 31 ימים
4. אפריל: 30 ימים
5. מאי: 31 ימים
6. יוני: 30 ימים
7. יולי: 31 ימים
8. אוגוסט: 31 ימים
9. ספטמבר: 30 יום
10. אוקטובר: 31 ימים
11. נובמבר: 30 ימים
12. דצמבר: 31 ימים

שנה מעוברת היא שנה המתחלקת ב-4 ללא שארית,
אך לא מתחלקת ב-100,
או
שנה המתחלקת ב-400 ללא שארית.

תאריך חוקי צריך לעמוד במספר כללים יסודיים:

✓ **שנה חוקית** – מספר שלם חיובי

✓ **חודש חוקי** – מספר שלם בין 1 ל-12

✓ **יום חוקי** – מספר שלם חיובי, ותלוי במספר הימים בחודש.

תקינות תאריך נדרשת בכל מקום שנדרש תאריך :
תאריך לידה, תאריך הזמנה, תאריך תשלום, האם תאריך קודם
לאחר, כמה ימים יש בין תאריך אחד לאחר ועוד.

כתבו פעולה המקבלת שלושה מספרים שלמים:

day, month, year

המחזירה אמת אם הם מהווים תאריך חוקי ושקר אחרת.

דוגמה מסכמת : תקינות תאריך

```
static void IsValidDateBadExample()
{
    Console.Write("Enter a day: ");
    int day = int.Parse(Console.ReadLine());

    Console.Write("Enter a month: ");
    int month = int.Parse(Console.ReadLine());

    Console.Write("Enter a year: ");
    int year = int.Parse(Console.ReadLine());

    bool dateOk = true;
```

```
if (year<1||month<1||month>12) dateOk = false;
else
{
    if (month==1||month==3||month==5|| month==7||
        month==8||month==10||month== 12)
    {
        if (day<1||day>31) dateOk = false;
    }
    else if (month==4||month==6||month==9||month==11)
    {
        if (day<1&&day>30) dateOk = false;
    }
    else
    {
        if ((year%4==0&&year%100!=0)|| (year%400==0))
        {
            if (day< 1||day>29) dateOk = false;
        }
        else
        {
            if (day<1&&day>28) dateOk = false;
        }
    }
}
```

```
Console.Write ($"The date {day}/{month}/{year} ");
if (dateOk) Console.WriteLine ("is valid.");
else Console.WriteLine ("is not valid.");
}
```

דוגמה מסכמת : תקינות תאריך

```
static void IsValidDateBadExample()
{
    Console.WriteLine("Enter a day: ");
    int day = int.Parse(Console.ReadLine());

    Console.WriteLine("Enter a month: ");
    int month = int.Parse(Console.ReadLine());

    Console.WriteLine("Enter a year: ");
    int year = int.Parse(Console.ReadLine());

    bool dateOk = true;
```

במידה של קלט בתוך הפעולה,
לא יתאפשר אחר-כך טיפול בנתונים
המתקבלים כפרמטרים ולא מהקלט :

- נתונים אקראיים, Random
- נתונים המתקבלים מטופס,
- נתונים המתקבלים ממסד נתונים.


```

if (year < 1 || month < 1 || month > 12)
    dateOk = false;
else
{
    if (month == 1 || month == 3 || month == 5 || month == 7 ||
        month == 8 || month == 10 || month == 12)
    {
        if (day < 1 || day > 31)
            dateOk = false;
    }
    else if (month == 4 || month == 6 || month == 9 || month == 11)
    {
        if (day < 1 && day > 30)
            dateOk = false;
    }
    else
    {
        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        {
            if (day < 1 || day > 29)
                dateOk = false;
        }
        else
        {
            if (day < 1 && day > 28)
                dateOk = false;
        }
    }
}

```

דוגמה מסכמת : תקינות תאריך

הקוד מכיל בדיקת טווחים
בין מינימום למקסימום.
שיכפול קוד עם ערכים שונים.

```

if (year < 1 || month < 1 || month > 12)
    dateOk = false;
else
{
    if (month == 1 || month == 3 || month == 5 || month == 7 ||
        month == 8 || month == 10 || month == 12)
    {
        if (day < 1 || day > 31)
            dateOk = false;
    }
    else if (month == 4 || month == 6 || month == 9 || month == 11)
    {
        if (day < 1 && day > 30)
            dateOk = false;
    }
    else
    {
        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        {
            if (day < 1 || day > 29)
                dateOk = false;
        }
        else
        {
            if (day < 1 && day > 28)
                dateOk = false;
        }
    }
}

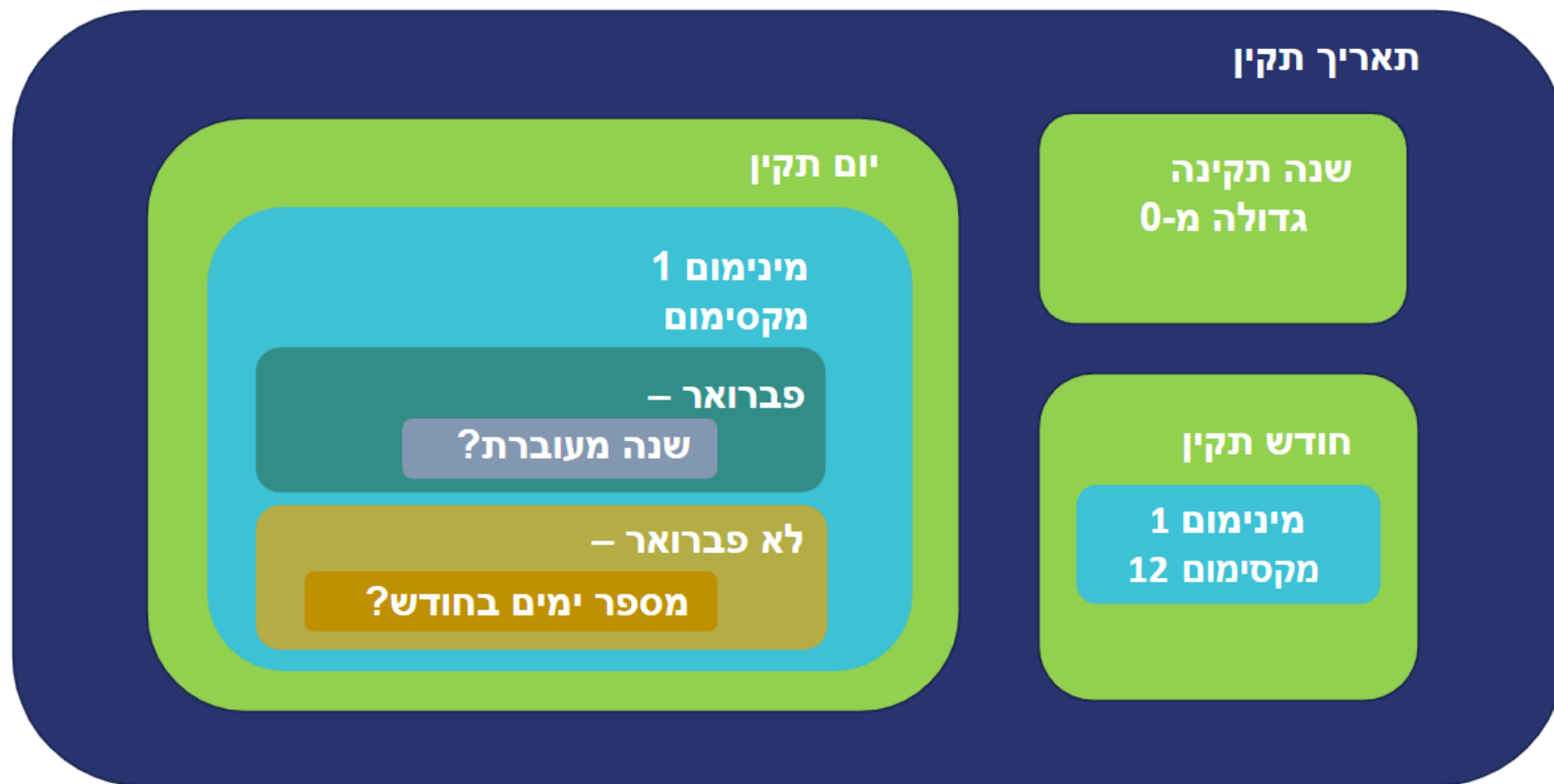
```

דוגמה מסכמת : תקינות תאריך

בדיקת שנה מעוברת.

לא נוכל להשתמש בבדיקה זו
לצורך משימות אחרות.
לדוגמה, הדפסת כל השנים
המעוברות בטווח שנים.

דוגמה מסכמת : תקינות תאריך



דוגמה מסכמת : תקינות תאריך

```
// הפעולה מקבלת שלושה מספרים
// הפעולה מחזירה אמת אם המספרים מהווים תאריך חוקי, שקר אחרת
static bool IsValidDate(int day, int month, int year)
{
    if (year < 1)
        return false;

    if (!InRange(1, 12, month))
        return false;

    if (month == 2)
        return InRange(1, MaxFebDays(year), day);

    return InRange(1, MaxDays(month), day);
}
```

הפעולה תחזיר את מספר הימים
בפברואר בשנה מסוימת.

הפעולה תחזיר את מספר הימים
בחודש מסוים.

הפעולה מקבלת :
- ערך תחתון,
- ערך עליון
- ומספר

ותבדוק אם המספר נמצא בטווח.

דוגמה מסכמת : תקינות תאריך – האם בטווח ?

יכולנו לוותר על min, כיוון שבבדיקת תאריך המינימום הוא תמיד 1,
אבל כך נוכל להשתמש בפעולה גם לצרכים אחרים

```
// הפעולה מקבלת שני מספרים שלמים - ערך תחתון וערך עליון  
// ומספר שלם  
// הפעולה מחזירה אמת אם המספר הוא בטווח המבוקש, שקר אחרת  
static bool IsInRange(int min, int max, int n)  
{  
    return n >= min && n <= max;  
}
```

דוגמה מסכמת : תקינות תאריך - מספר הימים בחודש

```
// הפעולה מקבלת מספר חודש
// הגבלה - מספר החודש אינו 2 (פברואר)
// הפעולה מחזירה את מספר הימים בחודש
static int MaxDays(int month)
{
    int d1 = 31, d2 = 30;
    if (month < 8)
    {
        if (month % 2 == 1) // 1,3,5,7
            return d1;
        else // 4,6
            return d2;
    }
    if (month % 2 == 0) // 8,10,12
        return d1;
    else // 9,11
        return d2;
}
```

```
// הפעולה מקבלת מספר חודש
// הגבלה - מספר החודש אינו 2 (פברואר)
// הפעולה מחזירה את מספר הימים בחודש
static int MaxDays(int month)
{
    int d = 30; // כל החודשים מלבד ...
                // 1,3,5,7
    if (month < 8 && month % 2 == 1) ||
        // 8,10,12
        (month >= 8 && month % 2 == 0)

        d = 31; // הם באורך 31 יום

    return d;
}
```

ובדרך אחרת (המשתמשת בתנאי מורכב)

דוגמה מסכמת : תקינות תאריך - מספר הימים בחודש פברואר

```
// הפעולה מקבלת מספר המייצג שנה
// הפעולה מחזירה את מספר הימים בפברואר בשנה זו
static int MaxFebDays(int year)
{
    if (IsLeapYear(year))
        return 29;
    return 28;
}
```

הפעולה תבדוק אם שנה מסוימת
היא שנה מעוברת

```
// הפעולה מקבלת מספר המייצג שנה
// הפעולה מחזירה אמת אם השנה היא שנה מעוברת, שקר אחרת
static bool IsLeapYear(int year)
{
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}
```

דוגמה מסכמת : תקינות תאריך

```
Random rnd = new Random();
int d, m, y;
for (int i = 0; i < 100; i++)
{
    d = rnd.Next(-10, 40);
    m = rnd.Next(-4, 15);
    y = rnd.Next(-300, 2300);

    Console.Write ($"The date {d}/{m}/{y} ");
    if (IsValidDate(d, m, y))
        Console.WriteLine ("is valid.");
    else
        Console.WriteLine ("is not valid.");
}
```

```
The date -10/6/795 is not valid.
The date 10/14/1077 is not valid.
The date -9/1/558 is not valid.
The date 1/14/1861 is not valid.
The date -9/11/990 is not valid.
The date 17/13/1219 is not valid.
The date 29/6/1414 is valid.
The date -7/7/2156 is not valid.
The date 21/13/84 is not valid.
The date 25/-4/2109 is not valid.
The date 1/5/2008 is valid.
The date -9/14/23 is not valid.
The date 9/7/-112 is not valid.
The date 36/7/1916 is not valid.
The date 0/12/1665 is not valid.
The date 26/7/880 is valid.
The date 15/-3/893 is not valid.
The date 14/10/1174 is valid.
The date -10/13/2191 is not valid.
The date 13/10/-83 is not valid.
The date 9/2/304 is valid.
The date 10/-4/822 is not valid.
The date 37/-4/1855 is not valid.
The date 8/-1/513 is not valid.
The date 15/11/2168 is valid.
The date 30/12/1610 is valid.
The date -1/13/537 is not valid.
The date 24/-1/2225 is not valid.
The date 13/9/1641 is valid.
The date 6/-4/-215 is not valid.
The date -2/13/1861 is not valid.
The date 39/6/432 is not valid.
The date 21/11/1987 is valid.
The date 6/10/1208 is valid.
The date 21/-2/2035 is not valid.
The date -3/14/2111 is not valid.
The date 20/-1/-81 is not valid.
The date 1/1/1956 is valid.
The date 28/9/28 is valid.
The date 3/13/394 is not valid.
The date 24/7/2282 is valid.
```


דוגמה מסכמת : תקינות תאריך

```
Random rnd = new Random();
int d, m, y;
for (int i = 0; i < 100; i++)
{
    d = rnd.Next(-10, 40);
    m = rnd.Next(-4, 15);
    y = rnd.Next(-300, 2300);

    Console.Write ($"The date {d}/{m}/{y} ");
    if (IsValidDate(d, m, y))
        Console.WriteLine ("is valid.");
    else
        Console.WriteLine ("is not valid.");
}
```

```
The date 29/2/2021 is not valid.
The date 30/2/2025 is not valid.
The date 27/2/2024 is valid.
The date 30/2/2025 is not valid.
The date 30/2/2027 is not valid.
The date 28/2/2025 is valid.
The date 28/2/2026 is valid.
The date 27/2/2030 is valid.
The date 30/2/2021 is not valid.
The date 28/2/2021 is valid.
The date 30/2/2023 is not valid.
The date 29/2/2026 is not valid.
The date 30/2/2027 is not valid.
The date 29/2/2021 is not valid.
The date 28/2/2030 is valid.
The date 30/2/2023 is not valid.
The date 30/2/2020 is not valid.
The date 29/2/2023 is not valid.
The date 30/2/2021 is not valid.
The date 27/2/2023 is valid.
The date 29/2/2020 is valid.
The date 28/2/2030 is valid.
The date 28/2/2026 is valid.
The date 28/2/2029 is valid.
The date 29/2/2030 is not valid.
The date 30/2/2022 is not valid.
The date 29/2/2020 is valid.
The date 27/2/2023 is valid.
The date 29/2/2022 is not valid.
The date 29/2/2024 is valid.
The date 30/2/2029 is not valid.
The date 27/2/2030 is valid.
The date 30/2/2021 is not valid.
The date 29/2/2023 is not valid.
The date 30/2/2027 is not valid.
The date 27/2/2026 is valid.
The date 30/2/2021 is not valid.
The date 28/2/2028 is valid.
The date 30/2/2028 is not valid.
The date 27/2/2022 is valid.
The date 28/2/2027 is valid.
```

שאלת סיכום : עידוד קניית ספרים

בחנות ספרים ערכו מבצע לעידוד קניית ספרים.

חלק מהספרים סומנו במבצע "ספר שני ב 50%" - לקוח שקונה 2 ספרים בסימון זה משלם חצי מחיר בעבור הספר הזול מביניהם. הספרים האחרים סומנו במבצע "ספר שלישי – הזול בחינם" - לקוח שקונה 3 ספרים מקבל במתנה את הספר הזול מביניהם.

א. כתבו פעולה המקבלת שלושה מספרים ממשיים גדולים מ-0, המייצגים מחירי ספרים, ומחזירה את הסכום של שני המספרים הגדולים מביניהם.

ב. כתבו פעולה שתקבל שני מספרים ממשיים גדולים מ-0 ושונים זה מזה, שכל אחד מהם מייצג מחיר של ספר. הפעולה תחזיר את הסכום שישלם הלקוח שקנה במבצע את שני הספרים שאלה מחיריהם.

ג. כתבו פעולה שתקבל את מספר המבצע - מספר זה יכול להיות 2 או 3. לכל מספר אחר, הפעולה תדפיס "ERROR" ותחזיר 999-. הפעולה תקלוט את מחירי הספרים שקנה, ותחזיר את הסכום שעליו לשלם על פי תנאי המבצע. הפעולה תדפיס לכל לקוח שקנה ספרים במעל 200 ש"ח - קיבלת תו של 50 שח לקניה הבאה. [חובה להשתמש בפעולות מסעיף א+ב.](#)

ד. ביום מסוים השתתפו במבצע 142 לקוחות. כתבו תכנית שתקלוט, בעבור כל לקוח שהשתתף במבצע, את מספר הספרים שקנה. התכנית תחשב ותדפיס את הסכום הכולל (לאחר הנחות) שעל כל לקוח לשלם בעבור כל הספרים שקנה, וכן אם זכאי לתו שי. בנוסף הפעולה תדפיס את המחיר הממוצע לספר שנמכר באותו יום. [חובה להשתמש בפעולה מסעיף ג.](#)

שאלת סיכום : עידוד קניית ספרים - פתרון שלב א'

בחנות ספרים ערכו מבצע לעידוד קניית ספרים.

חלק מהספרים סומנו במבצע "ספר שני ב 50%" - לקוח שקונה 2 ספרים בסימון זה משלם חצי מחיר בעבור הספר הזול מביניהם. הספרים האחרים סומנו במבצע "ספר שלישי – הזול בחינם" - לקוח שקונה 3 ספרים מקבל במתנה את הספר הזול מביניהם.

א. כתבו פעולה המקבלת שלושה מספרים ממשיים גדולים מ-0, המייצגים מחירי ספרים, ומחזירה את הסכום של שני המספרים הגדולים מביניהם.

```
// הפעולה מקבלת שלושה מספרים ממשיים
// הפעולה מחזירה את סכום שני המספרים הגדולים ביניהם
static double BooksPrice(double book1, double book2, double book3)
{
    double min = Math.Min(book1, book2);
    min = Math.Min(book3, min);
    return book1 + book2 + book3 - min;
}
```

שאלת סיכום : עידוד קניית ספרים - פתרון שלב ב'

בחנות ספרים ערכו מבצע לעידוד קניית ספרים.

חלק מהספרים סומנו במבצע "ספר שני ב 50%" - לקוח שקונה 2 ספרים בסימון זה משלם חצי מחיר בעבור הספר הזול מביניהם. הספרים האחרים סומנו במבצע "ספר שלישי – הזול בחינם" - לקוח שקונה 3 ספרים מקבל במתנה את הספר הזול מביניהם.

ב. כתבו פעולה שתקבל שני מספרים ממשיים גדולים מ-0 ושונים זה מזה, שכל אחד מהם מייצג מחיר של ספר. הפעולה תחזיר את הסכום שישלם הלקוח שקנה במבצע את שני הספרים שאלה מחיריהם.

```
// הפעולה מקבלת שני מספרים ממשיים
// הפעולה מחזירה את סכום במספר הגדול וחצי המספר השני
static double BooksPrice(double book1, double book2)
{
    double min = Math.Min(book1, book2);
    double max = Math.Max(book1, book2);

    return max + min / 2;
}
```

שאלת סיכום : עידוד קניית ספרים - שלב ג'

בחנות ספרים ערכו מבצע לעידוד קניית ספרים.

חלק מהספרים סומנו במבצע "ספר שני ב 50%" - לקוח שקונה 2 ספרים בסימון זה משלם חצי מחיר בעבור הספר הזול מביניהם.
הספרים האחרים סומנו במבצע "ספר שלישי - הזול בחינם" - לקוח שקונה 3 ספרים מקבל במתנה את הספר הזול מביניהם.

ג. כתבו פעולה שתקבל את מספר המבצע - מספר זה יכול להיות 2 או 3. לכל מספר אחר, הפעולה תדפיס "ERROR" ותחזיר 999-.
הפעולה תקלוט את מחירי הספרים שקנה, ותחזיר את הסכום שעליו לשלם על פי תנאי המבצע.
הפעולה תדפיס לכל לקוח שקנה ספרים במעל 200 ש"ח - קיבלת תו של 50 שח לקניה הבאה. [חובה להשתמש בפעולות מסעיף א+ב.](#)

```

static double CalculatePrice(int number)
{
    if (number < 2 || number > 3)
    {
        Console.WriteLine("ERROR");
        return -999;
    }
    double a, b, c, result;
    Console.Write("1st book price: ");
    a = double.Parse(Console.ReadLine());
    Console.Write("2nd book price: ");
    b = double.Parse(Console.ReadLine());
    if (number == 2)
        result = BooksPrice(a, b);
    else
    {
        Console.Write("3rd book price: ");
        c = double.Parse(Console.ReadLine());
        result = BooksPrice(a, b, c);
    }
    if (result > 200)
        Console.WriteLine("You received a gift of NIS 50 for your next purchase");
    return result;
}

```

שאלת סיכום : עידוד קניית ספרים

```

// הפעולה מקבלת מספר מבצע
// אם המספר אינו 2 או 3, הפעולה תדפיס שגיאה ותחזיר -999
//
// אחרת, תקלוט את מחירי הספרים שניקנו
// תחשב ותחזיר את הסכום לתשלום ע"פ המבצע
//
// אם סכום הקניה גדול מ-200
// הפעולה תדפיס כי הקונה זכאי לתו של 50 ש"ח

```

שאלת סיכום : עידוד קניית ספרים – שלב ד'

בחנות ספרים ערכו מבצע לעידוד קניית ספרים.

חלק מהספרים סומנו במבצע "**ספר שני ב 50%**" - לקוח שקונה 2 ספרים בסימון זה משלם חצי מחיר בעבור הספר הזול מביניהם.
הספרים האחרים סומנו במבצע "**ספר שלישי – הזול בחינם**" - לקוח שקונה 3 ספרים מקבל במתנה את הספר הזול מביניהם.

ד. ביום מסוים השתתפו במבצע 142 לקוחות. כתבו תכנית שתקלוט, בעבור כל לקוח שהשתתף במבצע, את מספר הספרים שקנה. התכנית תחשב ותדפיס את הסכום הכולל (לאחר הנחות) שעל כל לקוח לשלם בעבור כל הספרים שקנה, וכן אם זכאי לתו שי. בנוסף הפעולה תדפיס את המחיר הממוצע לספר שנמכר באותו יום. [חובה להשתמש בפעולה מסעיף ג.](#)

שאלת סיכום : עידוד קניית ספרים

פתרון שלב ד'

```
static void Main(string[] args)
{
    int noOfBooks, count = 0; ;
    double price, sum = 0;
    for (int k = 1; k <= 142; k++)
    {
        Console.Write("enter noOfBooks books : ");
        noOfBooks = int.Parse(Console.ReadLine());
        if (noOfBooks == 2 || noOfBooks == 3)
        {
            count += noOfBooks;
            price = CalculatePrice(noOfBooks);
            sum += price;
            Console.WriteLine("You have to pay " + price + " shekels.");
        }
    }
    double avg = sum / count;
    Console.WriteLine("Average price for one book: " + avg + " shekels.");
}
```


העברת פרמטרים by value (העשרה)

```
static void IncNumber(int x)
{
    Console.WriteLine("before increment: x=" + x);
    x++;
    Console.WriteLine("after increment: x=" + x);
}

static void Main(string[] args)
{
    int num = 3;

    Console.WriteLine("before function: num=" + num);
    IncNumber(num);
}
```

int: x	3
--------	---

הזיכרון של IncNumber

int: num	3
----------	---

הזיכרון של Main

before function: num=3
before increment: x=3

העברת פרמטרים by value (העשרה)

```
static void IncNumber(int x)
{
    Console.WriteLine("before increment: x=" + x);
    x++;
    Console.WriteLine("after increment: x=" + x);
}

static void Main(string[] args)
{
    int num = 3;

    Console.WriteLine("before function: num=" + num);
    IncNumber(num);
}
```

int: x	4
--------	---

הזיכרון של IncNumber

int: num	3
----------	---

הזיכרון של Main

```
before function: num=3
before increment: x=3
after increment: x=4
```

העברת פרמטרים by value (העשרה)

```
static void IncNumber(int x)
{
    Console.WriteLine("before increment: x=" + x);
    x++;
    Console.WriteLine("after increment: x=" + x);
}
```

```
static void Main(string[] args)
{
    int num = 3;
```

```
    Console.WriteLine("before function: num=" + num);
    IncNumber(num);
    Console.WriteLine("after function: num=" + num);
}
```

int: num	3
----------	---

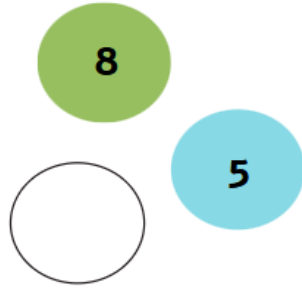
הזיכרון של Main

```
before function: num=3
before increment: x=3
after increment: x=4
after function: num=3
```

כאשר מעבירים פרמטר לפעולה למעשה מועבר
עותק של המשתנה המקורי לפעולה.
כלומר, אם הפרמטר השתנה בפעולה,
המשתנה המקורי **לא שונה**.

העברה כזו נקראת **by value**

החלפת ערכים (העשרה)



- נתונים עיגולים במרחב
- בשני עיגולים יש מספרים

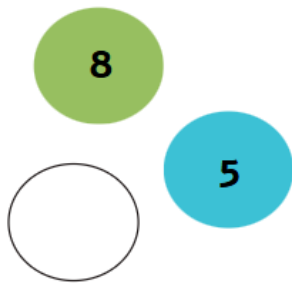
• המטרה:

להחליף בין ערכי העיגולים (כלומר, שהערך שבתכלת יהיה בירוק והערך שבירוק יהיה בתכלת)

• כללים:

- ✓ ערך חייב להיות בתוך עיגול
- ✓ בכל עיגול יהיה ערך אחד בלבד בכל פעם
- ✓ כדי שניתן יהיה להשתמש בערך מסויים הוא חייב להיות מאוחסן בעיגול

החלפת ערכים : אסטרטגיית הפתרון (העשרה)



✓ העבר ערכו של העיגול הירוק לעיגול הלבן

✓ העבר ערכו של העיגול התכלת לעיגול הירוק

✓ העבר ערכו של העיגול הלבן לעיגול התכלת

העברת פרמטרים by value (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(int x, int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(num1, num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

int: num1	5
int: num2	7

הזיכרון של Main

```
--> Main: num1=5, num2=7
```

העברת פרמטרים by value (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(int x, int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(num1, num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

```
--> Main: num1=5, num2=7
--> Swap: x=5, y=7
```

int: x	5
int: y	7
int: temp	5

הזיכרון של Swap

int: num1	5
int: num2	7

הזיכרון של Main

העברת פרמטרים by value (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(int x, int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(num1, num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

```
--> Main: num1=5, num2=7
--> Swap: x=5, y=7
```

int: x	7
int: y	5
int: temp	5

הזיכרון של Swap

int: num1	5
int: num2	7

הזיכרון של Main

העברת פרמטרים by value (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(int x, int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(num1, num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

int: num1	5
int: num2	7

הזיכרון של Main

```
--> Main: num1=5, num2=7
--> Swap: x=5, y=7
<-- Swap: x=7, y=5
```

העברת פרמטרים by value (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(int x, int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(num1, num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

int: num1	5
int: num2	7

הזיכרון של Main

הפעולה לא באמת החליפה בין 2 הערכים...
הערכים הועברו לפעולה ByVal
כלומר מה שעבר לפעולה זה רק עותק של הערכים.

```
--> Main: num1=5, num2=7
--> Swap: x=5, y=7
<-- Swap: x=7, y=5
<-- Main: num1=5, num2=7
```

העברת פרמטרים by reference (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(ref int x, ref int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(ref num1, ref num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

int: num1	5
int: num2	7

הזיכרון של Main

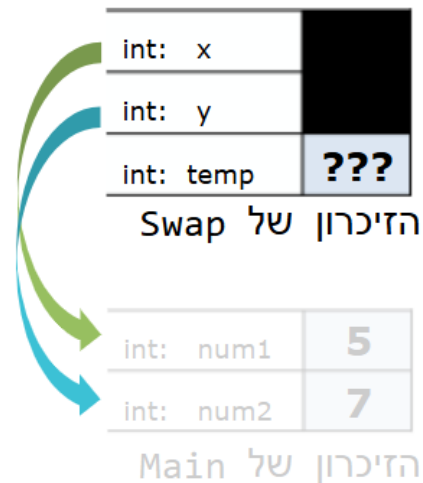
```
--> Main: num1=5, num2=7
```

העברת פרמטרים by reference (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(ref int x, ref int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(ref num1, ref num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

```
--> Main: num1=5, num2=7
```

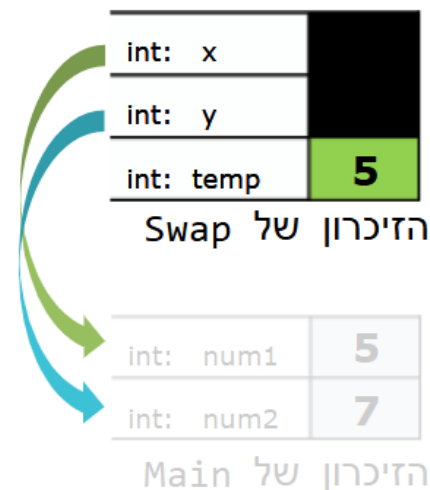


העברת פרמטרים by reference (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(ref int x, ref int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(ref num1, ref num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

```
--> Main: num1=5, num2=7
--> Swap: x=5, y=7
```

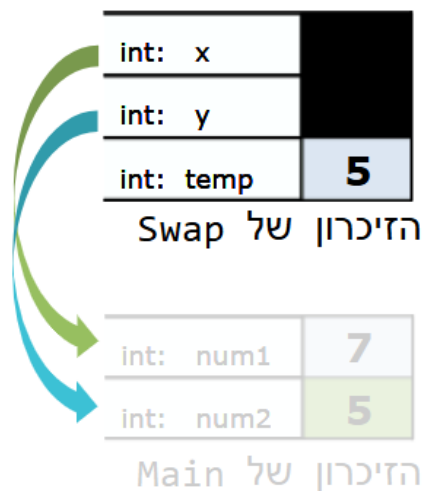


העברת פרמטרים by reference (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(ref int x, ref int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(ref num1, ref num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

```
--> Main: num1=5, num2=7
--> Swap: x=5, y=7
<-- Swap: x=7, y=5
```



העברת פרמטרים by reference (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(ref int x, ref int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(ref num1, ref num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

int: num1	7
int: num2	5

הזיכרון של Main

```
--> Main: num1=5, num2=7
--> Swap: x=5, y=7
<-- Swap: x=7, y=5
```

העברת פרמטרים by reference (העשרה): החלפת ערכי הפרמטרים

```
static void Swap(ref int x, ref int y)
{
    Console.WriteLine("--> Swap: x=" + x + ", y=" + y);
    int temp = x;
    x = y;
    y = temp;
    Console.WriteLine("<-- Swap: x=" + x + ", y=" + y);
}

static void Main(string[] args)
{
    int num1 = 5, num2 = 7;
    Console.WriteLine("--> Main: num1=" + num1 + ", num2=" + num2);
    Swap(ref num1, ref num2);
    Console.WriteLine("<-- Main: num1=" + num1 + ", num2=" + num2);
}
```

int: num1	7
int: num2	5

הזיכרון של Main

```
--> Main: num1=5, num2=7
--> Swap: x=5, y=7
<-- Swap: x=7, y=5
<-- Main: num1=7, num2=5
```


העברת פרמטרים by reference (העשרה)

- ראינו שניתן להעביר לפעולה **הפניות** למשתנים המקורים, ולא רק את הערכים שלהם
- העברה כזו של משתנים נקראת **by reference**
✓ בעזרת העברת משתנים by ref פתרנו את הבעיה של הפעולה Swap.
- מבחינת תחביר:
✓ כאשר פעולה רוצה לקבל משתנה **by ref** תצוין לפני שם הפרמטר המילה **ref**.
✓ כאשר מעבירים משתנה לפעולה שמקבלת פרמטר by reference גם יש לציין לפני ההעברה את המילה **ref**.
✓ אין להעביר לפעולה משתנה לא מאותחל, גם אם משתמשים ב-**ref**.

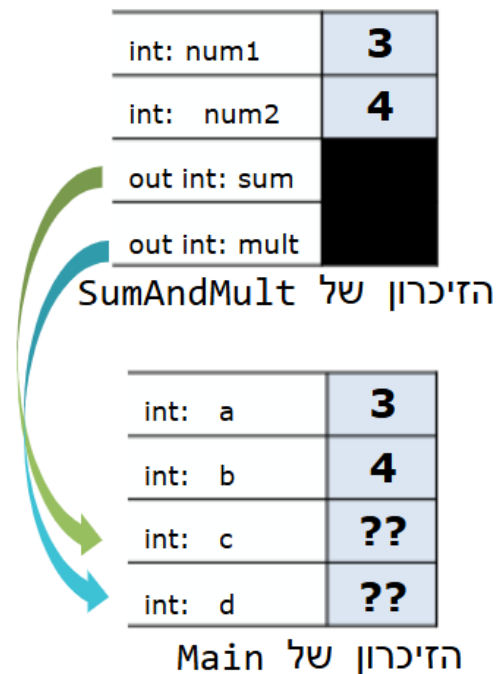
פעולה המחזירה יותר מערך אחד : **משתנה out** (העשרה)

- עד כה ראינו שפעולה יכולה להחזיר ערך יחיד.
- בעזרת שימוש ב-ref הפעולה יכולה להחזיר יותר מערך אחד ע"י השמה של הערכים המוחזרים במשתנה שהועבר עם ref.
- כאשר נרצה **לחייב** את הפעולה שתבצע השמה לתוך אחד הפרמטרים, לפני השימוש בו, או לפני שתחזיר אותו נכתוב לפניו, במימוש ובשימוש (זימון), את המילה **out**.
- משתנה out הוא כמו משתנה ref פרט לכך שאין חובה לאתחל אותו לפני השימוש.

משתנה out : דוגמה (העשרה)

```
static void SumAndMult (int num1, int num2, out int sum, out int mult)
{
    sum = num1 + num2;
    mult = num1 * num2;
}

static void Main(string[] args)
{
    int a=3, b=4, c, d;
    SumAndMult(a, b, out c, out d);
}
```



משתנה out : דוגמה (העשרה)

```
static void SumAndMult (int num1, int num2, out int sum, out int mult)
{
    sum = num1 + num2;
    mult = num1 * num2;
}

static void Main(string[] args)
{
    int a=3, b=4, c, d;
    SumAndMult(a, b, out c, out d);
}
```

int: num1	3
int: num2	4
out int: sum	
out int: mult	

הזיכרון של SumAndMult

int: a	3
int: b	4
int: c	7
int: d	12

הזיכרון של Main

משתנה out : דוגמה (העשרה)

```
static void SumAndMult (int num1, int num2, out int sum, out int mult)
{
    sum = num1 + num2;
    mult = num1 * num2;
}
```

```
static void Main(string[] args)
{
    int a=3, b=4, c, d;
    SumAndMult(a, b, out c, out d);
    Console.WriteLine(a + "+" + b + "=" + c);
}
```

int: a	3
int: b	4
int: c	7
int: d	12

הזיכרון של Main

משתנה out : דוגמה (העשרה)

```
static void SumAndMult (int num1, int num2, out int sum, out int mult)
{
    sum = num1 + num2;
    mult = num1 * num2;
}
```

```
static void Main(string[] args)
{
    int a=3, b=4, c, d;
    SumAndMult(a, b, out c, out d);
    Console.WriteLine(a + "+" + b + "=" + c);
    Console.WriteLine(a + "*" + b + "=" + d);
}
```

3+4=7
3*4=12

int: a	3
int: b	4
int: c	7
int: d	12

הזיכרון של Main

ביחידה זו למדנו:

- מהי פעולה ומיקומה במחלקה.
- פעולות ממחלקות אחרות (קופסא שחורה).
- טווח הכרה של משתנים.
- מחסנית הקריאות.
- פרמטרים של פעולה.
- פעולה המקבלת יותר מפרמטר אחד, כולל תוכנית לדוגמא.
- פעולה המחזירה ערך, כולל תוכנית לדוגמא.
- כיצד נראה הזיכרון.
- העמסת פעולות.
- תכנון Top-Down בשילוב פעולות כולל דוגמאות.
- שאלת סיכום.
- העברת פרמטרים.
- **העשרה:** פרמטרי ref ופרמטרי out.